

# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación



## Clase 1: Conceptos Desarrollo software e Ingeniería Software

Catedrático: Claudia Alegre

17 de enero de 2026

# Dinámica de Introducción

Conocer al equipo

- Catedrático se presenta  
Breve hoja de vida
- Alumnos se presentan  
Nombre,  
Estudiante/trabajador  
Expectativas del curso

# Tema

- Desarrollo de software profesional
- Ingeniería de software
- Diversidad de la ingeniería de software
- Ingeniería de software y la Web

# Indicadores de Logro

- El estudiante entenderá los conceptos básicos para iniciar proyectos

# Actividades

- Se explicará el objetivo del curso dentro de la carrera
- Validación de conocimientos
- Tarea 1

# Teoría: Desarrollo de software profesional

## **Desarrollo de software profesional**

- Es la práctica de diseñar, programar, probar y mantener aplicaciones informáticas con estándares de calidad y buenas prácticas.

### Características principales:

- Orientado a la satisfacción del cliente
- Uso de metodologías
- Trabajo en equipo multidisciplinario
- Documentación
- Control de versiones

# Teoría: Ingeniería de software:

## Ingeniería de software:

- “Es el establecimiento y uso de principios solidos de la ingeniería para obtener económicamente un software confiable y que funcione de modo eficiente en máquinas reales”. [Bauer, 1972]
- “Es el estudio de los principios y metodologías para desarrollo y mantenimiento de sistemas de software”. [Zelkovitz, 1978]
- “Es la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo operación (funcionamiento) y mantenimiento del software: es decir, la aplicación de ingeniería al software”. [IEEE, 1993]
- “Es una disciplina de la ingeniería que comprende todos los aspectos de la producción de software desde las etapas iniciales de la especificación del sistema hasta el mantenimiento de este después que se utiliza”. [Sommerville, 2004]
- “Es una disciplina que integra el proceso, los métodos, y las herramientas para el desarrollo de software de computadora”. [Pressman, 2005]

# Teoría: Diversidad de la ingeniería de software

## **Diversidad de la ingeniería de software**

Variedad de experiencias, conocimientos, perspectivas y habilidades que aportan los integrantes de un equipo.

Los equipos diversos ofrecen una serie de beneficios:

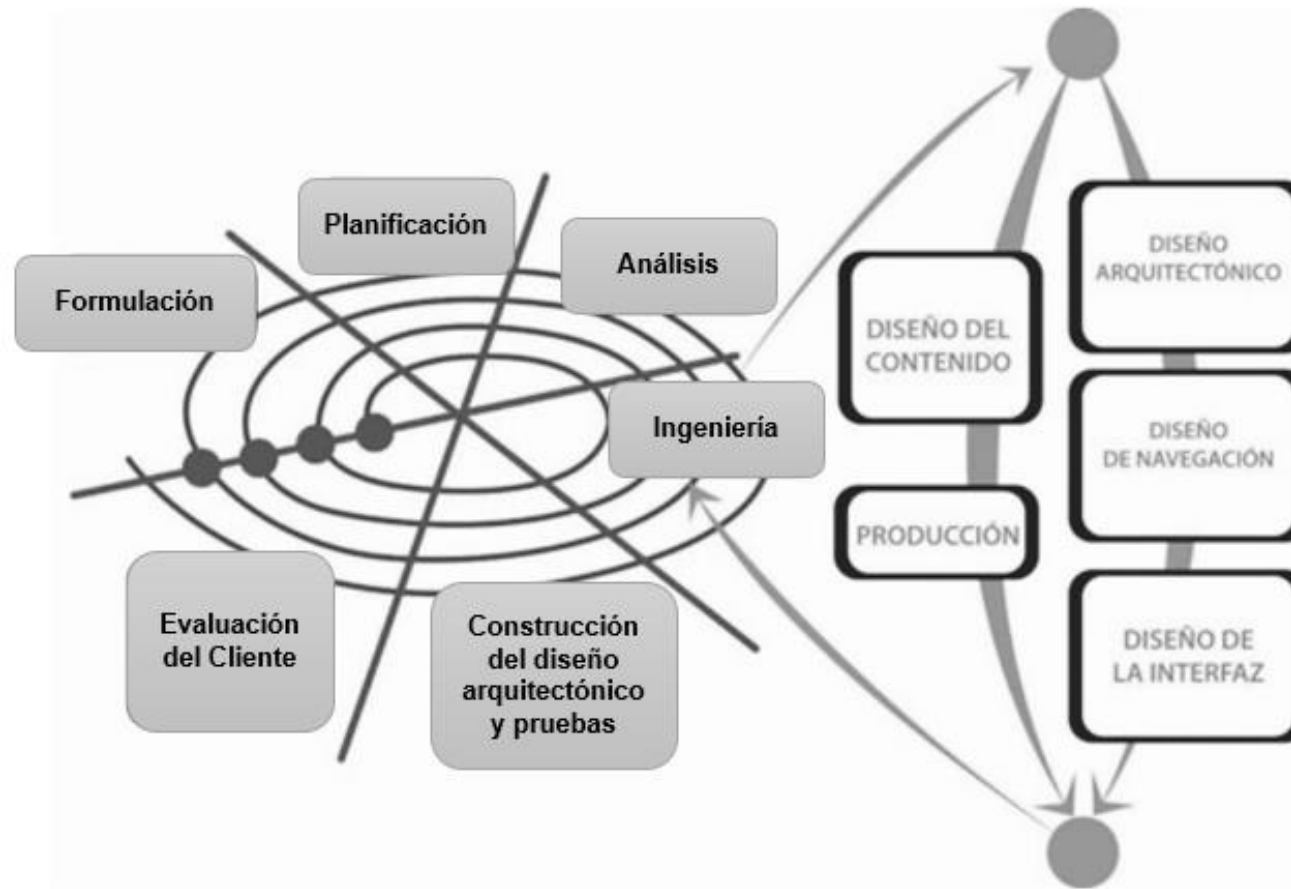
- Mayor creatividad e innovación
- Mejor discernimiento de las necesidades de los usuarios
- Reducción del sesgo

Google: Google tiene una política de diversidad e inclusión que se centra en la contratación, el desarrollo y la retención de empleados de diferentes orígenes. Como resultado de esta política, Google ha sido reconocido como uno de los mejores lugares para trabajar para las mujeres y las minorías.

Microsoft: Microsoft ha implementado una serie de iniciativas de diversidad e inclusión, entre las que se incluyen programas de tutoría y mentoría para empleados de minorías. Como resultado de estas iniciativas, Microsoft ha visto un aumento en la diversidad de su fuerza laboral.



# Teoría: Ingeniería de Software y la Web



# Actividad en Grupos

¿Por qué enmarcar un proyecto tecnológico dentro de una metodología?

# Antes de Concluir la Sesión

Cuál fue el punto más importante que vimos en la clase de hoy?  
Qué metodologías de implementación y desarrollo de proyectos has utilizado?

¿Qué es la ingeniería de software y algunos ejemplos? –  
MejorSoftware Blog

Qué me interesó de esta lectura?

# Tarea para el Viernes Próximo

Los entregables de un proyecto de sistema de información

# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación



## Clase 2: Conceptos Desarrollo software e Ingeniería Software

Catedrático: Claudia Alegre

24 de enero de 2026

# Dinámica de Introducción

Los entregables de un proyecto

<https://youtu.be/FjlzfXbk7dA>

# Tema

- Modelos de proceso de software
- El modelo en cascada
- Desarrollo incremental
- Ingeniería de software orientada a la reutilización
- Actividades del proceso
- Especificación del software Diseño e implementación del software
- Validación de software
- Evolución del software

# Indicadores de Logro

- Comprensión de los procesos de software
- Entender qué es un proceso de software y su importancia en el desarrollo de sistemas.
- Identifica herramientas y técnicas utilizadas en la ingeniería de software para implementar modelos de proceso



# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea 1

# Teoría: Modelos de proceso de software

Son enfoques sistemáticos que describen los pasos y las actividades del ciclo de vida de desarrollo de software (SDLC).

- **Modelo en cascada: El enfoque secuencial tradicional**
- **Modelo ágil: Flexibilidad y colaboración**
- **Modelo en espiral: Un enfoque iterativo y basado en el riesgo**
- **Modelos iterativos e incrementales: Desarrollo evolutivo**

# Teoría: Modelo en cascada

Modelo de desarrollo de software donde el trabajo avanza en fases secuenciales:

- Requisitos
- Diseño
- Implementación
- Pruebas
- Despliegue
- Mantenimiento.

Cada etapa debe completarse antes de pasar a la siguiente, sin retrocesos. Este enfoque funciona mejor en proyectos con requisitos estables y bien definidos desde el inicio.

# Teoría: Modelo en cascada

- **Requisitos:** Se definen necesidades del cliente, alcance y restricciones del sistema.
- **Diseño:** Se diseña la arquitectura, componentes, bases de datos e interfaces.
- **Implementación:** Se programa el software siguiendo el diseño aprobado.
- **Pruebas:** Se valida que el sistema cumple especificaciones y funciona correctamente.
- **Despliegue:** Se instala el sistema en el entorno productivo del cliente.
- **Mantenimiento:** Se corrigen errores y se hacen ajustes controlados.

# Teoría: Modelo en cascada

- **Cascada pura:** Secuencia rígida sin solapamientos ni retrocesos entre fases.
- **Cascada con retroalimentación limitada:** Permite volver solo a la fase anterior para corregir errores puntuales.
- **Cascada en V:** Relaciona cada fase de desarrollo con una fase de prueba correspondiente, formando una “V”.
- **Cascada con prototipos:** Añade construcción de prototipos tempranos para validar requisitos o interfaces.
- **Cascada iterativa controlada:** Repite el ciclo completo en versiones grandes, pero manteniendo fases secuenciales en cada iteración.

# Teoría: Modelo Ágil

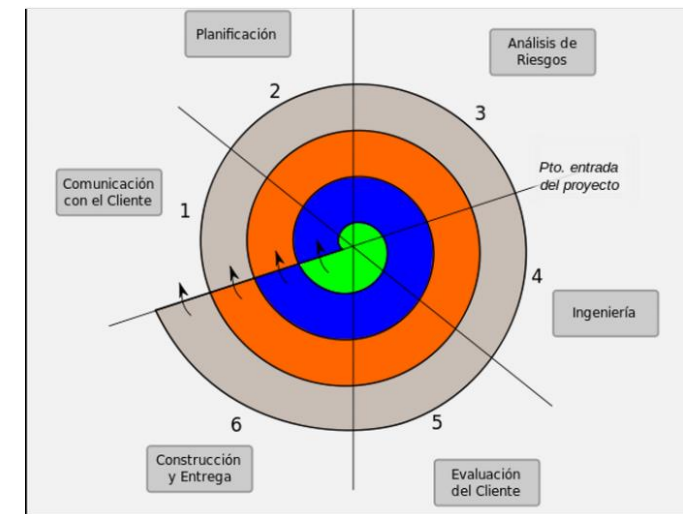
Conjunto de técnicas aplicadas en ciclos de trabajo cortos, con el objetivo de que el proceso de entrega de un proyecto sea más eficiente.

Hagamos grupos de trabajo....

[Manifiesto por el Desarrollo Ágil de Software](#)

# Teoría: Modelo en espiral

- El modelo de desarrollo en Espiral es una combinación entre el modelo waterfall y un modelo por iteraciones.
- El proceso pasa por distintas etapas, desde la de conceptualización, siguiendo el desarrollo, luego una fase de mejoras, para finalizar con el mantenimiento.



# Teoría: Modelo iterativo e incremental

En un desarrollo iterativo e incremental el proyecto se planifica en diversos **bloques temporales** (en el caso de Scrum de un mes natural o hasta de dos semanas, si así se necesita) llamados iteraciones.

En cada iteración el equipo **evoluciona el producto** (hace una entrega incremental) a partir de los resultados completados en las iteraciones anteriores, añadiendo nuevos objetivos/requisitos o mejorando los que ya fueron completados.



# Teoría: Modelo iterativo / espiral

| Aspecto    | Modelo Iterativo                           | Modelo Espiral                                                  |
|------------|--------------------------------------------|-----------------------------------------------------------------|
| Objetivo   | Mejorar el producto en ciclos              | Reducir riesgos en cada ciclo                                   |
| Estructura | Iteraciones sucesivas                      | Fases en espiral (planificación, riesgos, desarrollo, revisión) |
| Riesgos    | Poco énfasis                               | Enfoque central                                                 |
| Uso típico | Proyectos medianos, con cambios frecuentes | Proyectos grandes, críticos o de alto riesgo                    |
| Costo      | Menor                                      | Mayor (por análisis de riesgos)                                 |

# Teoría: Ingeniería de software orientada a la reutilización

Los componentes de software que implementan un objeto bien definido o función pueden ser reutilizados.

Beneficios:

- Reduce el riesgo de procesos
- Uso efectivo de especialistas
- Cumplimiento de estándares
- Desarrollo acelerado

Problemas:

- Incremento en costos de mantenimiento
- Falta de herramientas de soporte
- No fue inventado aquí...

# Teoría: Especificaciones del software, Diseño e implementación de software

## ANALISIS DE REQUERIMIENTOS ( ESPECIFICACIONES )

Específicamente, esta fase se refiere a:

- El **dominio de aplicación**, es decir, el contexto en el que el software está destinado a operar.
- La **información sobre la aplicación**, como su propósito, definiciones, descripción general del sistema, referencias.
- Las **funciones que realizará**, características de los usuarios, restricciones o posibles dependencias.
- Los **requisitos de rendimiento**.
- Las **especificaciones de la interfaz de usuario**.
- Los **requisitos de la base de datos**.

**Esta es la fase más importante de la ingeniería de software**, ya que cometer un error en este momento puede provocar pérdidas de tiempo desagradables, con el riesgo de volver a abrir temas ya abordados y cerrados anteriormente.

# Teoría: Especificaciones del software, Diseño e implementación de software

## **Diseño**

Fase en la que se definen las **estructuras de datos, las funciones y los comportamientos** en función de las restricciones impuestas por los requisitos protagonistas de la fase anterior. En la práctica, se define la arquitectura y la estructura de los componentes individuales del sistema.

## **Implementación. Transforma el diseño en algo funcional**

Después de haber concebido un diseño detallado del software, comienza la etapa de Implementación, donde cada línea de código cuenta. Aquí es donde realmente damos vida a las ideas, transformando esos planos y maquetas digitales en un programa que la gente pueda usar.

# Teoría: Validación y evolución de software

## **Validación:**

La verificación y validación es el nombre que se da a los procesos de comprobación y análisis que aseguran que el software que se desarrolla está acorde a su especificación y cumple las necesidades de los clientes

## **Evolucion:**

Proceso continuo de mejora y adaptación , no es estático, necesita adaptarse a los cambios tecnológicos, de mercado y de usuarios.

# Antes de Concluir la Sesión

Cuál fue el punto más importante que vimos en la clase de hoy?

Qué modelo de proceso de software te interesó más?

# Tarea para el Viernes Próximo

Diseño modular, investigación

# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación



## Clase 3: Conceptos Desarrollo software e Ingeniería Software

Catedrático: Claudia Alegre

24 de enero de 2026

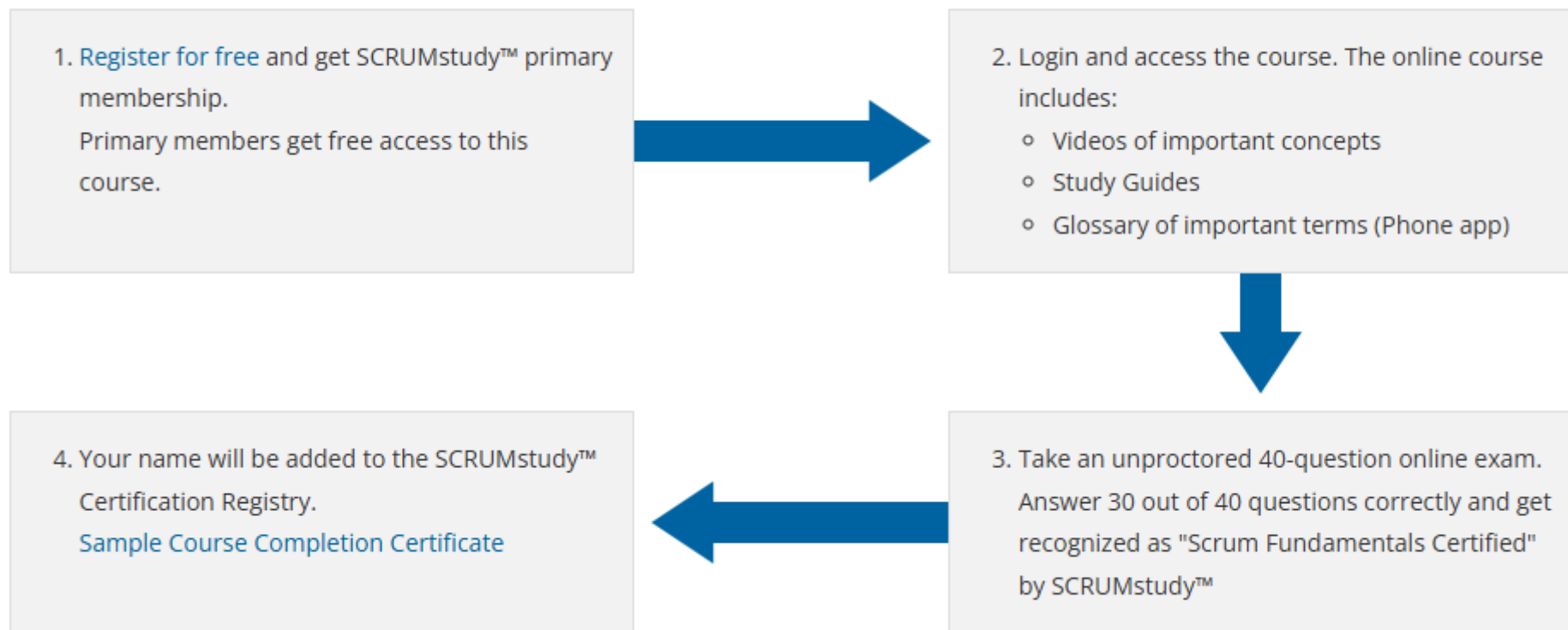


# Dinámica de Introducción

Hablemos de la certificación Scrum Fundamentals...

<https://www.scrumstudy.com/certification/scrum-fundamentals-certified>

## How to Apply



**Stakeholder:** no entiendo por qué una funcionalidad tan simple toma tanto tiempo para ser implementada.

DEUDA TÉCNICA

A photograph of a utility worker in a blue shirt and white hard hat, seen from behind, working on a wooden utility pole. The pole is completely obscured by a dense, chaotic, and massive tangle of black cables and wires that hang down and spread out in all directions. The background shows a building with windows. A blue diagonal banner with the text 'DEUDA TÉCNICA' is overlaid on the image. A white text box at the top contains a quote from a stakeholder.

# Tema

- Desarrollo ágil de software.
- Métodos ágiles Desarrollo dirigido por un plan y desarrollo ágil
- Programación extrema
- Pruebas en XP
- Programación en pares
- Administración de un proyecto ágil

# Indicadores de Logro

- Comprensión del Desarrollo Ágil.
- Explica los principios fundamentales del Manifiesto Ágil, incluyendo los valores y principios clave.
- Identifica los roles clave en un equipo ágil, como Product Owner, Scrum Master y Development Team.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea 1

# Teoría: Desarrollo ágil de software

A medida que la sociedad evolucionaba, y la tecnología con ella, muchos desarrolladores y programadores se percataron de que el modelo en cascada resultaba muy tedioso.

Estos proyectos se dividen en partes más pequeñas.

El equipo de desarrolladores trabajan en ellas de forma progresiva, dividiendo cada una de las partes en otras más reducidas, priorizando y agrupando tareas. Cada vez que completan una de las partes, los desarrolladores deben tener un producto que enviar al cliente, de modo que este lo pueda probar

Se basa en seis pasos fundamentales, que cubren la totalidad del ciclo de vida del programa:

- Planificación
- Estudio de requisitos
- Diseño
- Codificación
- Evaluación
- Documentación.
- En cada una de las interacciones, el equipo responsable del desarrollo no entrega la totalidad del software.

# Teoría: Métodos ágiles Desarrollo dirigido por un plan y desarrollo ágil

El desarrollo ágil se centra en

- la flexibilidad
- la colaboración
- la retroalimentación del cliente
- la iteración rápida



# Teoría: Programación Extrema

La Programación Extrema (XP) se centra en la entrega continua de valor, la excelencia técnica y la mejora constante del equipo.

## Valores Fundamentales

XP se basa en cinco valores esenciales que guían el trabajo del equipo:

- Comunicación: Promueve discusiones cara a cara para resolver problemas de manera eficiente.
- Simplicidad: Se enfoca en hacer solo lo necesario, evitando complejidad innecesaria.
- Retroalimentación: Fomenta ajustes continuos basados en comentarios sobre el diseño y la implementación.
- Coraje: Impulsa a enfrentar problemas y probar nuevas soluciones cuando algo no funciona.
- Respeto: Los miembros del equipo deben valorarse mutuamente y trabajar en colaboración.

# Teoría: Programación Extrema

## Principios Clave

- Retroalimentación rápida
- la simplicidad
- el cambio incremental
- la adaptación al cambio
- el trabajo de calidad.

# Teoría: Programación Extrema

## Prácticas Destacadas

XP propone prácticas que combinan planificación, organización y desarrollo técnico:

- **Juego de Planificación:** Involucra al cliente en la planificación para priorizar tareas según valor y costo.
- **Pequeñas Entregas:** Libera versiones iterativas del software con frecuencia.
- **Desarrollo Guiado por Pruebas (TDD):** Escribe pruebas antes del código para garantizar calidad.
- **Programación en Parejas:** Dos desarrolladores trabajan juntos en una misma tarea para mejorar la calidad del código.
- **Integración Continua:** Integra y prueba el código frecuentemente para detectar errores rápidamente.
- **Refactorización:** Mejora el diseño del código sin alterar su funcionalidad.

# Teoría: Programación Extrema

## Aplicaciones Actuales

- Hoy en día, XP se utiliza como un marco flexible que puede combinarse con otras metodologías ágiles como Scrum o Kanban. Sus prácticas, como la integración continua y las historias de usuario, son fundamentales en enfoques modernos como DevOps y arquitecturas de microservicios.
- XP fomenta un entorno colaborativo donde el equipo entrega valor constante, responde rápidamente al feedback y promueve la mejora continua. Esto crea una cultura de excelencia técnica y motivación dentro del equipo.

# Teoría: Programación En pares

Técnica de desarrollo de software en la que dos programadores trabajan juntos en el mismo código al mismo tiempo.

Un programador, llamado conductor, escribe el código mientras que el otro, llamado navegador, revisa el código y proporciona comentarios.

Los roles se cambian con frecuencia para garantizar que ambos programadores participen activamente en el proceso.

La programación en pares se utiliza por varias razones como:

- Mejorar la calidad del código
- Mejorar la colaboración,
- Aumentar la productividad
- Facilitar el aprendizaje.

# Teoría: Programación En pares

## Ventajas de la programación en pares

- Mejor calidad del código

Por ejemplo, la programación en pares puede ayudar a elegir nombres de variables apropiados, evitar la duplicación de código y seguir el principio de mínimo asombro.

- Colaboración mejorada

Por ejemplo, la programación en pareja puede ayudar a incorporar nuevos desarrolladores, asesorar a desarrolladores junior y distribuir el conocimiento del dominio entre todo el equipo.

- Mayor productividad

Por ejemplo, la programación en pareja puede ayudar a cumplir los plazos , ofrecer funciones y alcanzar objetivos.

# Teoría: Programación En pares

## Desventajas

- Costo más alto

Por ejemplo, la programación en pareja puede afectar el presupuesto, el alcance y el cronograma del proyecto.

- Flexibilidad reducida

La programación en pareja puede limitar la flexibilidad y autonomía de los desarrolladores, ya que tienen que coordinar y sincronizar sus horarios, preferencias y estilos.

- Posibles conflictos

La programación en pareja puede generar conflictos y tensiones entre los desarrolladores, ya que tienen que trabajar estrecha e intensamente entre sí.

# Teoría: Administración de un proyecto ágil

Hay métodos que se utilizan para aplicar la metodología ágil:

## **Scrum:**

Marco hiper-organizado que pide a los equipos que completen sprints en un marco de tiempo rígido.

A través de una organización minuciosa de los miembros del equipo y los requisitos del proyecto, Scrum permite a su equipo completar los sprints de una manera oportuna al tiempo que garantiza mejoras para futuros proyectos.



# Teoría: XP -Scrum

La principal diferencia entre Scrum y XP radica en su enfoque y prácticas.

Scrum se centra en la gestión ágil de proyectos, XP se enfoca en las prácticas de desarrollo de software

Aunque comparten algunos principios, Scrum se centra en la gestión del proyecto, mientras que XP se enfoca en las prácticas de desarrollo de software.

# Teoría: Manifiesto ágil

El Agile Manifesto, Manifiesto ágil o Manifiesto para el desarrollo de software ágil es una declaración de valores y principios sobre nuevas formas de desarrollar software que surgió en 2001, como reacción a los tradicionales métodos formales con los que se trabajaba entonces en la industria.

# Teoría: Manifiesto ágil

## Los 4 valores del Manifiesto Agile

- Individuos e interacciones por encima de procesos y herramientas
- Software funcionando por encima de documentación exhaustiva
- Colaboración con el cliente por encima de negociación contractual
- Respuesta ante el cambio por encima de seguir un plan

# Teoría: Manifiesto ágil

## Los 12 principios de Agile

- Satisfacer al cliente mediante la entrega temprana y continua
- Aprovechar el cambio como ventaja competitiva
- Entregar valor frecuentemente
- Cooperación negocio-desarrolladores durante todo el proyecto
- Construir proyectos en torno a individuos motivados
- Utilizar la comunicación cara a cara
- Software funcionando como medida de progreso
- Promover y mantener un desarrollo sostenible
- La excelencia técnica mejora la agilidad
- La simplicidad es fundamental
- Equipos auto-organizados para generar más valor
- Reflexión y ajustes frecuentes del trabajo de los equipos

# Teoría: SCRUM

Marco de trabajo ágil diseñado para gestionar proyectos de manera eficiente, promoviendo la colaboración, la adaptabilidad y la entrega continua de valor.



# Teoría: SCRUM

## Roles principales

- **Product Owner:** Responsable de priorizar el backlog del producto y garantizar que el equipo trabaje en las tareas más importantes.
- **Scrum Master:** Facilita el proceso, elimina obstáculos y asegura que se sigan las prácticas de Scrum.
- **Equipo de Desarrollo:** Encargado de implementar y probar las funcionalidades del producto.

# Teoría: SCRUM

## Roles principales

- **Product Owner:** Responsable de priorizar el backlog del producto y garantizar que el equipo trabaje en las tareas más importantes.
- **Scrum Master:** Facilita el proceso, elimina obstáculos y asegura que se sigan las prácticas de Scrum.
- **Equipo de Desarrollo:** Encargado de implementar y probar las funcionalidades del producto.



# Antes de Concluir la Sesión

Equipo 1: Principios de Ágil.

Equipo 2: Principios de Cascada.

Equipo 3: Ventajas de Ágil.

Equipo 4: Ventajas de Cascada.

Equipo 5: Riesgos o limitaciones de ambos enfoques.

# Tarea para el Viernes Próximo

Porque SCRUM?



# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación

## Clase 4: Ingeniería de requerimientos

Catedrático: Claudia Alegre

7 de febrero de 2026

# Dinámica de Introducción

El problema.....

<https://youtu.be/4jEF2mwOiTw>

# Tema

- Requerimientos funcionales y no funcionales.
- El documento de requerimientos de software
- Especificación de requerimientos.
- Especificación en lenguaje natural.
- Especificaciones estructuradas

# Indicadores de Logro

Reconoce y distingue entre requerimientos funcionales y no funcionales en un proyecto de software.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Requerimientos funcionales ...

Esenciales para definir las capacidades y comportamientos específicos que debe tener un sistema.

Estos requerimientos detallan qué debe hacer el sistema para cumplir con las expectativas y necesidades de los usuarios finales, y se enfocan en las funciones y características tangibles que el software debe proporcionar.

Por ejemplo, en un sistema de gestión de inventarios, un requerimiento funcional podría ser la capacidad de agregar, modificar y eliminar registros de productos. Otro ejemplo podría ser la generación de informes de ventas mensuales o la posibilidad de buscar productos por nombre o código.

Estos requerimientos son cruciales porque determinan cómo interactuarán los usuarios con el software y qué operaciones podrán realizar.



# Teoría: ...Requerimientos funcionales

- Sirven como base para el diseño y la implementación del sistema.
- Ayudan a garantizar que el proyecto de software cumpla con las expectativas de los stakeholders y a evitar malentendidos durante el desarrollo.
- Son fundamentales para el proceso de pruebas, ya que permiten verificar si el sistema cumple con las especificaciones establecidas.
- En términos de documentación, los requerimientos funcionales suelen ser detallados en el documento de análisis de requisitos. Este documento incluye descripciones claras y precisas de cada función que debe realizar el sistema, así como los criterios de aceptación para cada requerimiento. La documentación debe ser lo suficientemente detallada como para que los desarrolladores y analistas puedan entender exactamente qué se espera del sistema, pero también debe ser accesible y comprensible para los stakeholders no técnicos.

# Teoría: Requerimientos no funcionales..

Aspectos esenciales que describen cómo debe comportarse un sistema de software, en lugar de las funciones específicas que debe realizar.

Estos requerimientos abordan características como el rendimiento, la seguridad, la usabilidad y la escalabilidad, y son fundamentales para garantizar la calidad global del proyecto de software.

En términos de rendimiento, los requerimientos no funcionales especifican aspectos como la rapidez de respuesta del sistema, la capacidad de procesamiento de datos y la eficiencia del uso de recursos.

# Teoría: ...Requerimientos no funcionales

Por ejemplo, un sistema de ventas en línea debe ser capaz de manejar miles de transacciones por segundo sin degradar su rendimiento.

Juegan un papel crucial en el análisis de requisitos, ya que garantizan que el sistema no solo funcione correctamente, sino que también ofrezca una experiencia de usuario óptima, segura y eficiente.

# En que me enfoco dentro de todos los requerimientos

Como priorizamos

<https://youtu.be/TRp3ZkjKtj8>

# Empecemos con un caso

Una pizzería local quiere implementar un **sistema en línea de pedidos**. El dueño pide:

- Que los clientes puedan ver el menú y ordenar pizzas.
- Que se pueda pagar en línea.
- Que el sistema avise a la cocina automáticamente.
- Que se guarden los datos de clientes frecuentes.

Listar requerimientos funcionales(5), requerimientos no funcionales(5)

Ahora clasifiquemos en

Obligatorios (sin ellos el sistema no funciona)

Deseables (mejoran la experiencia, pero no son críticos)

Opcionales (podrían añadirse en el futuro).

# Teoría: Especificación de requerimientos

## **Pasos:**

- Identificación de las necesidades del cliente
- Análisis de los Requerimientos
- Priorización y Validación de Requerimientos
- Documentación de Requerimientos

# Teoría:El documento de requerimientos de software

- **Introducción:** Se debe ofrecer una visión general del documento, incluyendo su propósito, alcance, definiciones, acrónimos y abreviaturas, y referencias. Esta sección sirve para contextualizar el contenido y asegurar que todos los lectores estén en la misma página respecto al vocabulario y objetivos del documento.
- **Descripción general del producto:** Aquí se presenta una visión macro del producto de software que se va a desarrollar, incluyendo una perspectiva de alto nivel de sus funciones y las interacciones con otros sistemas. También se describe el tipo de usuario y las restricciones generales.
- **Requerimientos específicos**

**Funcionales:** Deben detallarse con claridad cada una de las funciones que el software debe realizar. Esto incluye procesos, datos de entrada y salida, y la interacción con otros componentes.

**No Funcionales:** Factores como la seguridad, la escalabilidad, la performance, la usabilidad y la compatibilidad. Estos aspectos son cruciales para asegurar la calidad del producto final.

# Teoría:El documento de requerimientos de software

**Requerimientos de interfaz:** Esbozo de las interfaces de usuario, de hardware y de comunicación que el sistema deberá poseer.

**Requerimientos de datos:** Descripción de los datos necesarios, su estructura, almacenamiento y cómo deben ser manipulados.

**Restricciones de diseño:** Limitaciones impuestas sobre la solución de software debido a regulaciones, presupuesto, integraciones existentes, etc.

- Validación de requerimientos: Métodos y procedimientos que se utilizarán para confirmar que los requerimientos se han cumplido. Esto puede incluir casos de prueba, revisiones, entre otros mecanismos de verificación.
- Priorización y acuerdo de requerimientos: Es fundamental que el documento contemple un proceso de priorización de los requerimientos, así como los acuerdos a los cuales se llegó con los stakeholders respecto a la importancia de cada uno.



# Practiquemos

Sigamos con el caso de la pizzería, hagamos una plantilla simplificada de requerimientos:

- Introducción
- Alcance
- Requerimientos funcionales
- Requerimientos no funcionales
- Restricciones

# Teoría: Especificación en lenguaje natural

Desde los albores de la ingeniería de software, el lenguaje natural se usa para escribir los requerimientos de software.

Es expresivo, intuitivo y universal.

También es potencialmente vago, ambiguo y su significado depende de los antecedentes del lector.

Como resultado, hay muchas propuestas para formas alternativas de escribir los requerimientos.

Sin embargo, ninguna se ha adoptado de manera amplia, por lo que el lenguaje natural seguirá siendo la forma más usada para especificar los requerimientos del sistema y del software.

# Teoría: Mejores prácticas para definir los requisitos del software

- **Involucrar a las Partes Interesadas en Todo el Proceso**
- **Utilizar Métodos Ágiles para la Definición de Requisitos**
- **Utilizar Herramientas de Gestión de Requisitos del Software**
- **Realizar Pruebas de Aceptación del Usuario (UAT)**

# Teoría: Herramientas

La gestión de requisitos de software es esencial para el desarrollo de proyectos exitosos. Para apoyar este proceso, se utilizan diversas herramientas que facilitan la captura, análisis y seguimiento de los requisitos.

## 1. Casos de Uso

Los casos de uso representan una técnica fundamental en la ingeniería de requisitos, centrada en la identificación de las interacciones entre los usuarios y el sistema.

# Teoría: Herramientas

## 2. Diagramas de Flujo

Los diagramas de flujo son herramientas visuales que representan el flujo de procesos dentro de un sistema

## 3. Diagramas de Clase

Los diagramas de clase son parte fundamental de la modelación en la programación orientada a objetos

# Antes de Concluir la Sesión

Entrega tu documento de requerimientos

# Tarea para el Viernes Próximo

Lectura :

<https://www.soloingenieria.org/ingenieria-de-software/ieee-830-especificacion-de-requisitos/>

# Curso: Ingeniería de Software

## Ingeniería en Sistemas y Ciencias de la Computación

### Clase 5: Modelado de sistemas

Catedrático: Claudia Alegre

14 de febrero de 2026



# Dinámica de Introducción

Resuelve el cuestionario

# Tema

## **Modelado del sistema**

- Modelos de contexto
- Modelos de interacción
  - Modelado de casos de uso
  - Diagramas de secuencia
- Modelos estructurales
  - Diagramas de clase
  - Generalización
  - Agregación
- Modelos de comportamiento

# Indicadores de Logro

- Explica la importancia del modelado en el desarrollo de sistemas y su rol en la comprensión y comunicación de los requisitos.
- Identifica y describe los principales tipos de modelos de sistemas, como modelos estructurales, de comportamiento y de interacción.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Modelado de sistema

Un modelo es una representación abstracta y simplificada de un sistema

real, con la cual se puede explicar o probar su comportamiento como un todo o en partes.

A modo general, los modelos de sistemas de información representan:

Componentes

Propiedades de los componentes

Relaciones entre los componentes

# Teoría: Modelos de contexto

El diagrama de contexto, es una vista de alto nivel de un sistema. Es un medio básico para definir una entidad basándose en sus límites de alcance y en su relación con los componentes externos como las partes interesadas, proporcionando una representación profesional del sistema.

# Teoría: Modelos de contexto

Tres características claves.

- Producto – El proyecto, sistema o entidad a definir, se simboliza con un círculo que contiene en su interior todo lo que está bajo su control (por ejemplo, funciones y responsabilidades del trabajo, procesos).
- Entidades externas o agentes – Las personas, los sistemas y las organizaciones que funcionan por fuera pero que interactúan de alguna manera con el producto (es decir, los responsables de la toma de decisiones, los proveedores, los clientes).
- Líneas de flujo – Se representan mediante flechas y simbolizan los datos que fluyen entre los agentes o las formas específicas en que las entidades interactúan con el producto.

.

# Teoría: Modelos de contexto





# Teoría: Modelos de contexto

## CASO BIBLIOTECA:

Actualmente, la biblioteca maneja todo en papel:

- Los estudiantes deben llenar formularios para registrarse y pedir libros.
- Los bibliotecarios anotan manualmente los préstamos y devoluciones.
- Los profesores solicitan reservas de libros para sus cursos, pero no hay un sistema que lo organice.
- Los proveedores entregan nuevos libros y materiales, pero la información se guarda en carpetas físicas.
- La administración universitaria necesita reportes de uso, pero los bibliotecarios tardan semanas en preparar la información.

El nuevo sistema debe:

- Registrar estudiantes, profesores y bibliotecarios.
- Gestionar préstamos y devoluciones de libros.
- Permitir reservas de libros para cursos.
- Administrar inventario de libros y proveedores.
- Generar reportes automáticos para la administración.

Elabora un modelo de Contexto con este caso..

# Teoría: Modelos de Interacción

Todos los sistemas interactúan de alguna manera:

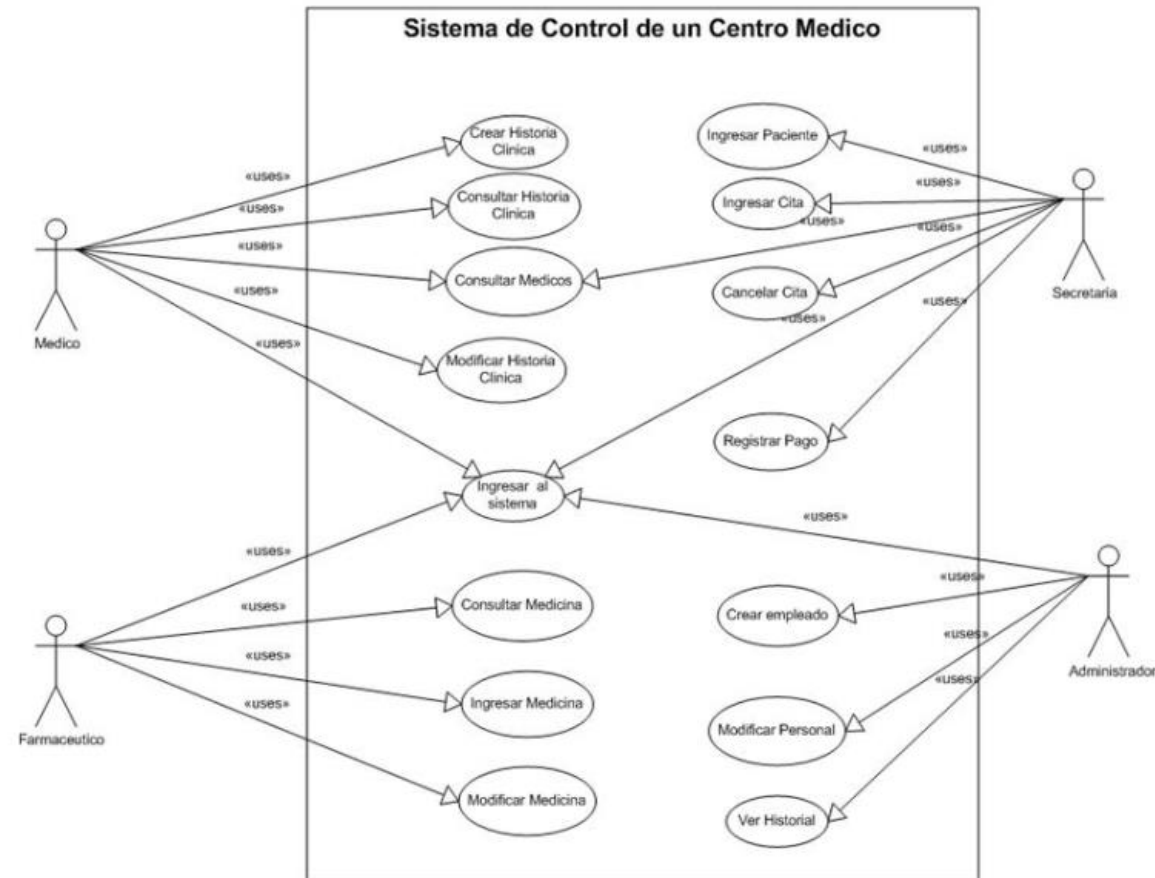
- Entradas y salidas del usuario
- Entre el sistema desarrollado y otros sistemas
- Entre componentes del sistema

Los más usados:

- Modelo de casos de uso
- Diagrama de secuencias

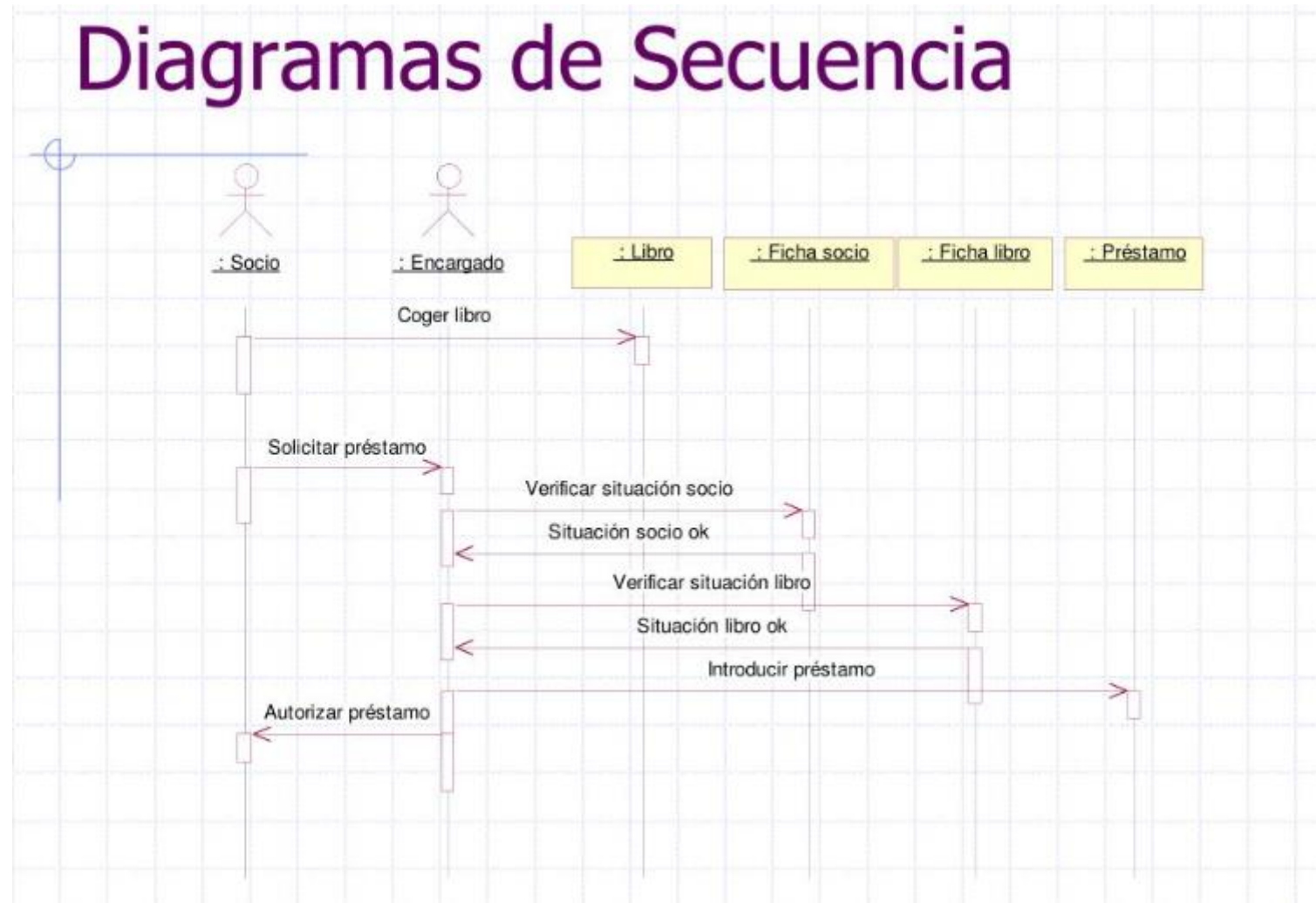
# Teoría: Modelos de Interacción

## Casos de uso



# Teoría: Modelos de Interacción

## Diagrama de secuencia



# Teoría: Modelos de contexto

CASO BIBLIOTECA:

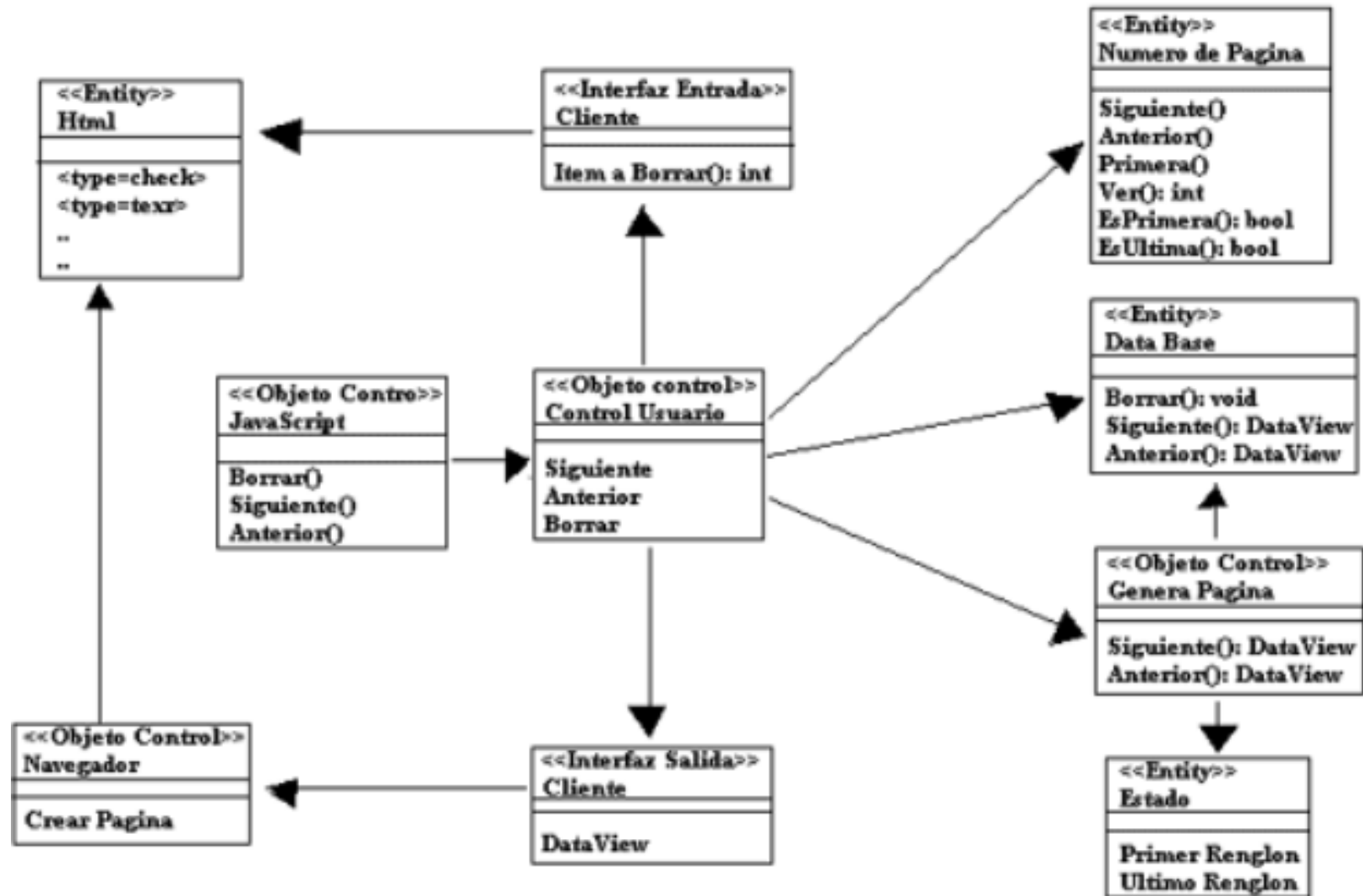
Elabora un modelo de Casos de uso

# Teoría: Modelos Estructurales

El modelado estructural sirve para describir los diferentes tipos y relaciones estáticas existentes entre los diferentes objetos de un sistema. Al igual que un arquitecto crea los planos de un edificio antes de su construcción, a la hora de desarrollar software, debemos modelar correctamente el sistema previamente al desarrollo, pudiendo incorporar el patrón o los patrones de diseño que nos ayuden a optimizar la solución a un determinado problema. La principal forma de emplear el modelado estructural es mediante el uso de:

- **Diagramas de clases:** normalmente están formados por Clases, Interfaces y Relaciones (de dependencia, de realización, de generalización, de asociación -agregación y composición-, etc).

# Teoría: Modelos Estructurales



# Teoría: Modelos Estructurales - Generalización

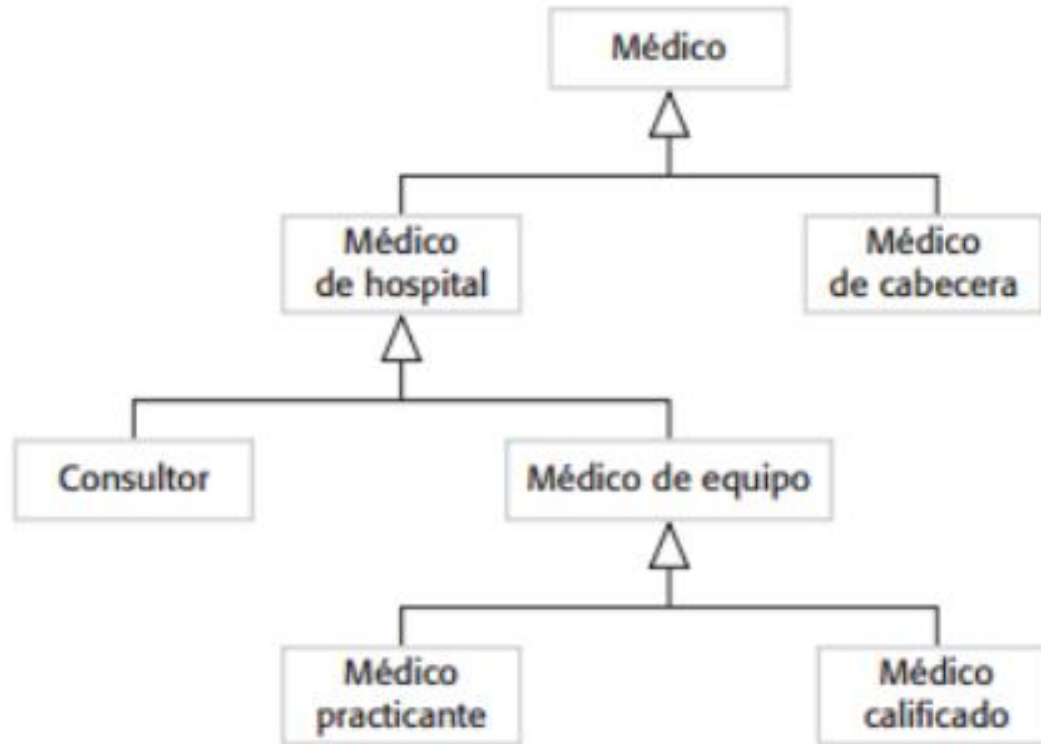
La generalización es una técnica cotidiana que se usa para gestionar la complejidad. En vez de aprender las características detalladas de cada entidad que se experimenta, dichas entidades se colocan en clases más generales (animales, automóviles, casas, etcétera) y se aprenden las características de dichas clases.

Esto permite deducir que diferentes miembros de estas clases tienen algunas características comunes (por ejemplo, las ardillas y ratas son roedores).

Es posible hacer enunciados generales que se apliquen a todos los miembros de la clase (por ejemplo, todos los roedores tienen dientes para roer). En el modelado de sistemas, con frecuencia es útil examinar las clases en un sistema, con la finalidad de ver si hay ámbito para la generalización. Esto significa que la información común se mantendrá solamente en un lugar. Ésta es una buena práctica de diseño, pues significa que, si se proponen cambios, entonces no se tiene que buscar en todas las clases en el sistema, para observar si se ven afectadas por el cambio. En los lenguajes orientados a objetos, como Java, la generalización se implementa usando los mecanismos de **herencia de clase** contruidos en el lenguaje.



# Teoría: Modelos Estructurales - Generalización

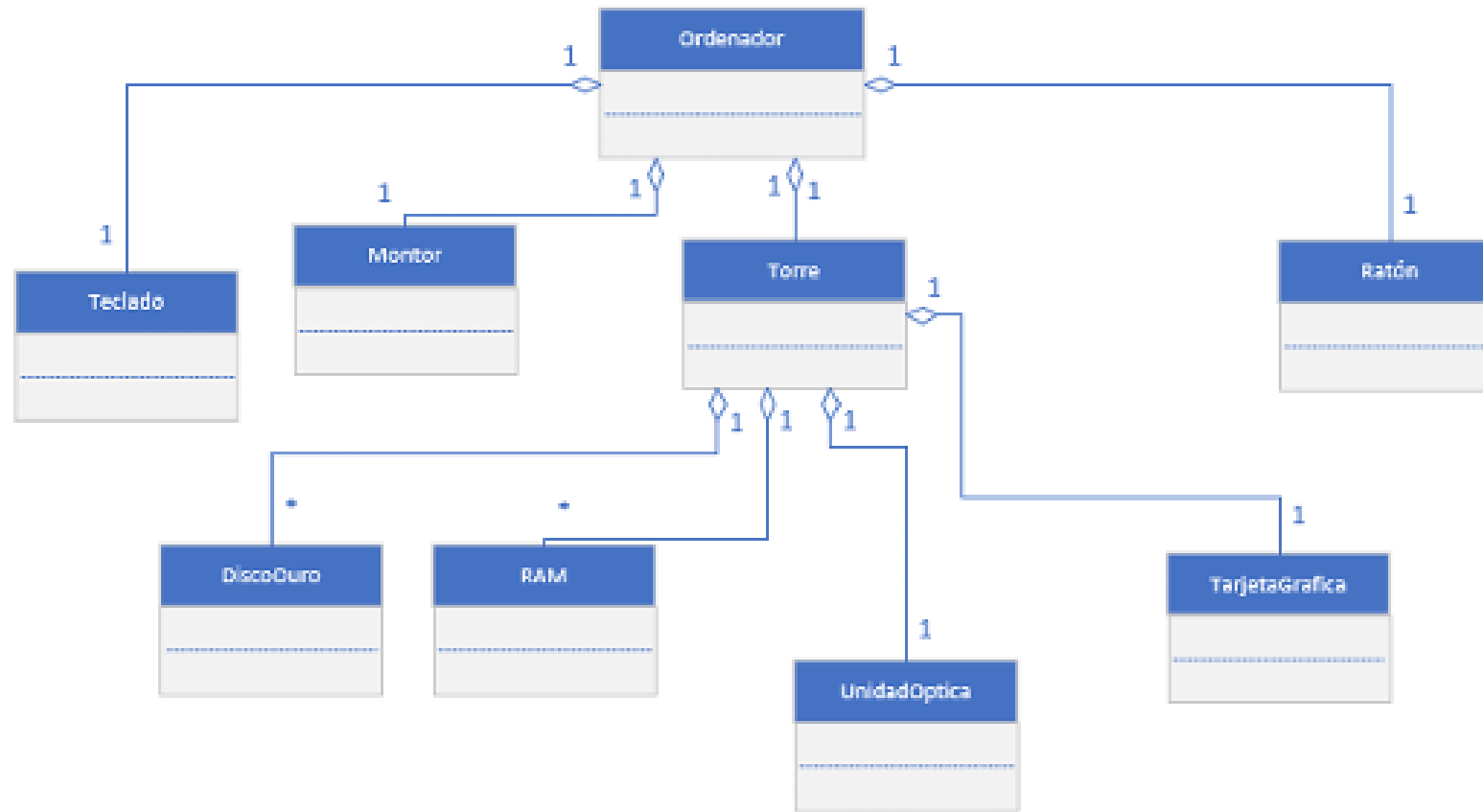


# Teoría: Modelos Estructurales - Agregación

“Una agregación se describe comúnmente como una relación “tiene un”. La agregación provee el medio para mostrar que todo el objeto está compuesto de la suma de sus partes”

(Kendall & Kendall, Análisis y diseño de sistemas, 2011, pág. 305).

# Teoría: Modelos Estructurales - Agregación



# Teoría: Modelos de Comportamiento

Los modelos de comportamiento se utilizan para describir el comportamiento del sistema en su totalidad.

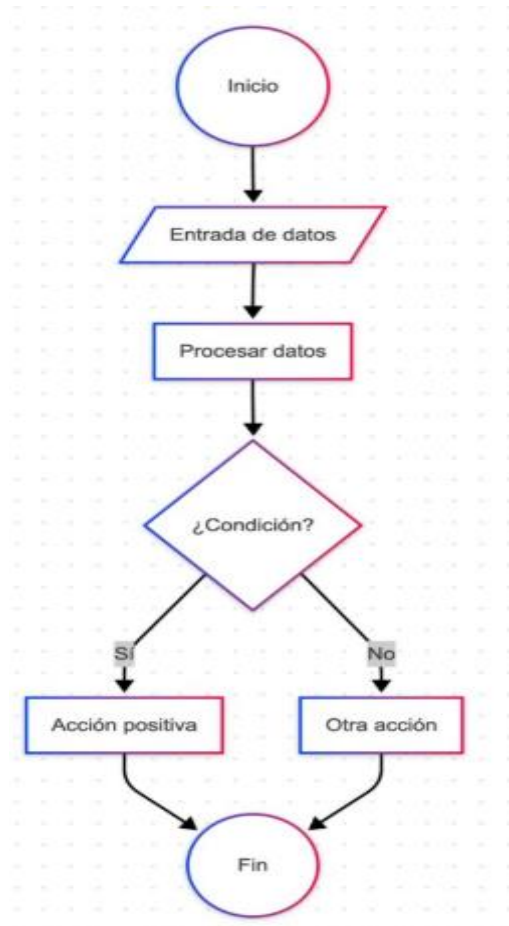
Entre los modelos de comportamiento existentes se distinguen dos de estos:

modelo de flujo de datos, que modelan el procesamiento de los datos en el sistema,

y modelos de máquinas de estado, que modelan como el sistema reacciona a los eventos.

Estos modelos pueden usarse de forma separada o conjuntamente, dependiendo del tipo de sistema que se esté desarrollando.

# Teoría: Modelos de Comportamiento



# Antes de Concluir la Sesión

Subir tus modelos realizados en clase

# Tarea para el Viernes Próximo

Estudiar



# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación

## Clase 7: Diseño Arquitectónico

Catedrático: Claudia Alegre

28 de febrero de 2026



# Dinámica de Introducción

Modelos de comportamiento.... Diagrama de flujo

INSCRIPCION UNIVERSITARIA:

El estudiante accede al portal de inscripción e ingresa su **carnet** y **contraseña**.

El sistema valida las credenciales:

- Si son incorrectas, muestra un mensaje de error y permite reintentar.
- Si son correctas, continúa.

El estudiante selecciona el **ciclo académico** y los **cursos disponibles**.

El sistema verifica:

- Si el curso tiene cupo disponible.
- Si el estudiante cumple los **requisitos previos**.
- Si no cumple, se muestra un mensaje de rechazo.

Si cumple, el curso se agrega a la lista de inscripción.

El estudiante confirma la inscripción y selecciona el **método de pago** (tarjeta, transferencia, efectivo).

El sistema valida el pago:

- Si es exitoso, genera un **comprobante digital** y actualiza la base de datos.
- Si falla, muestra un mensaje de error y permite reintentar.

Finalmente, se envía un **correo de confirmación** al estudiante con los cursos inscritos

# Tema

## **Diseño arquitectónico**

- Decisiones en el diseño arquitectónico
- Vistas arquitectónicas
- Patrones arquitectónicos
- Arquitectura en capas
- Arquitectura de repositorio
- Arquitectura cliente-servidor
- Microservicios

# Indicadores de Logro

- Explica los principios fundamentales del diseño arquitectónico en ingeniería de software, incluyendo modularidad, escalabilidad, y mantenibilidad.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Diseño Arquitectónico de Software

## DISEÑO ARQUITECTÓNICO

Una fase temprana del proceso de diseño del sistema.

Representa el vínculo entre los procesos de especificación y diseño.

Suelen llevarse a cabo en paralelo con las actividades de algunas especificaciones.

Se trata de identificar los principales componentes del sistema y sus comunicaciones.

# Teoría: Diseño Arquitectónico de Software

## DECISIONES DE DISEÑO ARQUITECTÓNICO

Diseño arquitectónico es un proceso creativo, por lo que el proceso es diferente dependiendo del tipo de sistema que se está desarrollado, antecedentes y experiencia del arquitecto.

Es común una serie de decisiones, en todos los procesos de diseño.

# Teoría: Vistas Diseño Arquitectónico de Software

## VISTAS ARQUITECTÓNICAS

**Vista Lógica:** Abstracciones como objetos o clases de objetos.

**Vista de Proceso:** Muestra como en tiempo de operación los componentes interactúan. (rendimiento y disponibilidad)

**Vista de Desarrollo:** Muestra como el software está descompuesto

**Vista Física:** Expone el hardware del sistema y como los componentes de software se distribuyen en los procesadores.

**Vista Abstracta:** Descomponer los requerimientos de alto nivel en especificaciones detalladas.

# Teoría: Diseño Arquitectónico de Software

## ESTRUCTURA DEL SISTEMA

Concerniente a la descomposición del sistema en sub-sistemas.

El diseño arquitectónico se expresa normalmente como un diagrama de bloques que presentan un panorama general de la estructura del sistema.

Modelos más específicos muestran cómo los sub-sistemas comparten los datos, se distribuyen y la interfaz con los demás también pueden ser desarrollados.



# Teoría: Modelos Arquitectónicos

## MODELOS ARQUITECTÓNICOS

1. Arquitectura en Capas
2. Arquitectura Cliente-Servidor
3. Arquitectura de Microservicios
4. Arquitectura de Capas N
5. Arquitectura Orientada a Servicios
6. Arquitectura Basada en Eventos
7. Arquitectura de Tubería y Filtro
8. Arquitectura de Espacio de Nombres
9. Arquitectura de Sistema Distribuido
10. Arquitectura Monolítica
11. Arquitectura de Microkernel
12. Arquitectura Hexagonal
13. Arquitectura basada en Componentes
14. Arquitectura de Peer-to-Peer
15. Arquitectura CQRS (Command Query Responsibility Segregation)
16. Arquitectura Basada en Nubes
17. Arquitectura de Repositorio
18. Modelo-Vista-Controlador
19. Arquitectura de Máquina de Estados
20. Arquitectura Orientada a Objetos
21. Arquitectura Broker (intermediario)
22. Arquitectura Master-Slave
23. Arquitectura Serverless

Al examinar cada Arquitectura plantea si es Jerárquico o No Jerárquico

Estos modelos proporcionan distintas formas de organizar y estructurar un sistema de software según los requisitos del proyecto. La elección del modelo depende de factores como la escalabilidad, mantenimiento, distribución, complejidad y el tipo de problema que se está resolviendo.

# Teoría: Modelos Arquitectónicos

## 1. ARQUITECTURA EN CAPAS

- Divide el sistema en capas o niveles donde cada capa tiene una responsabilidad específica.
- La comunicación entre capas se da de manera jerárquica: una capa superior utiliza los servicios de una capa inferior.
- Las capas de nivel inferior representan servicios núcleo que es probable se utilicen a lo largo de todo el sistema.

## 1. ARQUITECTURA EN CAPAS: VENTAJAS

- Separación de preocupaciones: Cada capa tiene una responsabilidad bien definida.
- Mantenibilidad: Los cambios en una capa no afectan directamente a otras capas.
- Reusabilidad: Las capas pueden ser reutilizadas en diferentes sistemas.

## 1. ARQUITECTURA EN CAPAS: USO

- Al construirse nuevas facilidades encima de los sistemas existentes.
- Cuando el desarrollo se dispersa a través de varios equipos de trabajo, y cada uno es responsable de una capa de funcionalidad.
- Cuando exista un requerimiento para seguridad multinivel.

## 1. ARQUITECTURA EN CAPAS: DESVENTAJAS

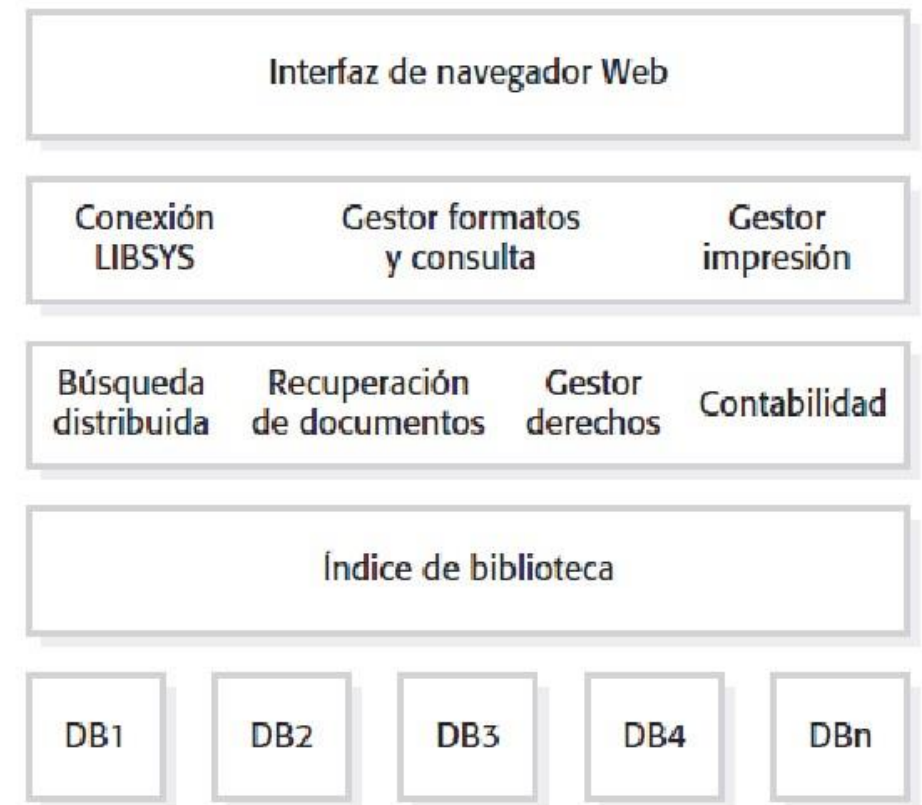
- Rendimiento: Puede haber overhead (sobrecarga) al pasar los datos a través de múltiples capas.
- Rigidez: Puede ser difícil cambiar una capa sin afectar a las demás.

# Arquitectura en capas, Ejemplo

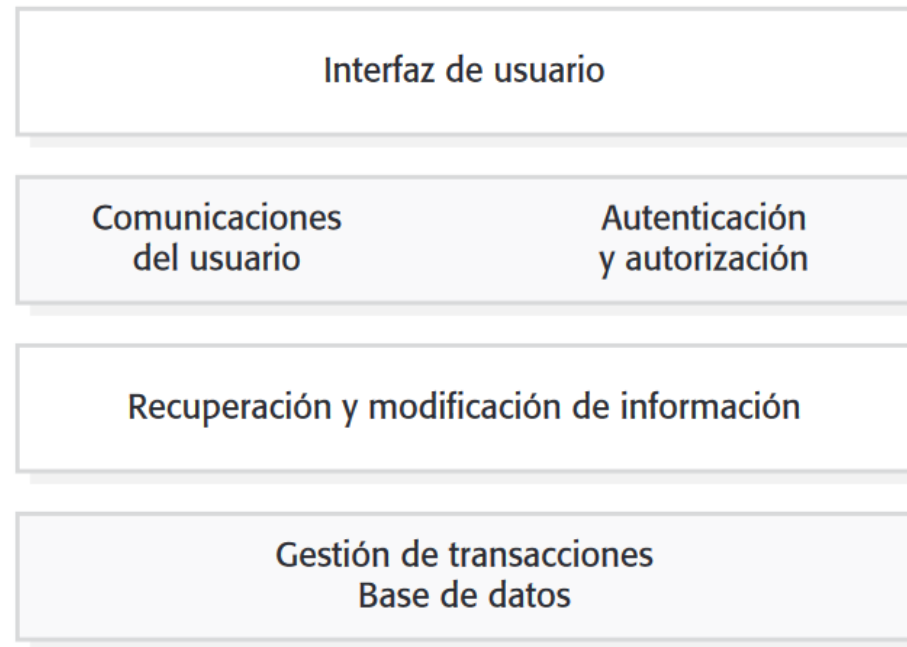
## 1. ARQUITECTURA EN CAPAS: EJEMPLO

Aplicaciones web que tienen capas de presentación, lógica de negocio y acceso a datos.

### Sistema de Biblioteca



# Arquitectura de sistema de información en capas



# Arquitectura Cliente Servidor

- Sistema distribuido que muestra cómo el modelo de datos y procesamiento se distribuye a través de una gama de componentes.
- Conjunto de servidores independientes que ofrecen servicios específicos, tales como la impresión, gestión de datos, etc
- Conjunto de clientes que envían solicitudes a éstos servidores, que los procesan y responden con los resultados.
- Red que permite a los clientes acceder a los servidores.

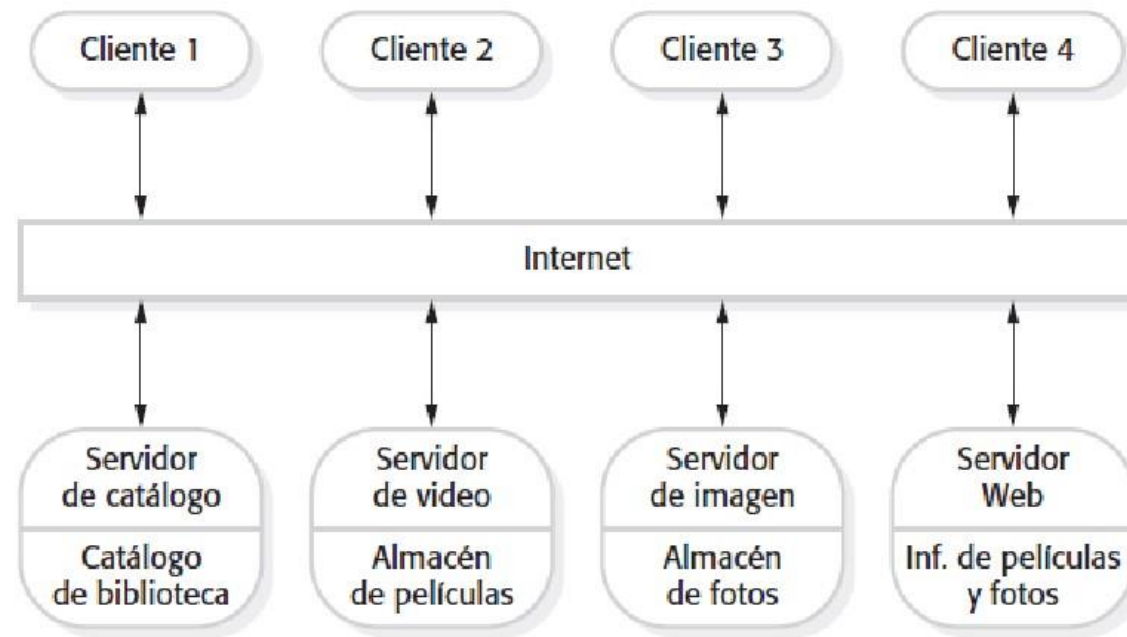
# Arquitectura Cliente Servidor

## Evolución

| Generación    | Características                              |
|---------------|----------------------------------------------|
| 1ª Generación | Servidores físicos dedicados, cliente pesado |
| 2ª Generación | Virtualización, cliente ligero               |
| 3ª Generación | Computación en la nube, SaaS                 |
| 4ª Generación | Microservicios, contenedores, orquestación   |

# Arquitectura Cliente Servidor

## 2. BIBLIOTECA DE IMÁGENES Y PELÍCULAS



# Arquitectura Cliente Servidor

## 2. ARQUITECTURA CLIENTE-SERVIDOR: VENTAJAS

- Centralización: El servidor central puede controlar y gestionar los datos y el acceso.
- Escalabilidad: Es más fácil escalar el servidor que administrar múltiples clientes.
- Mantenimiento: Es más sencillo de mantener o de trasladar con sencillez el servidor a nuevo hardware/software



# Arquitectura Cliente Servidor

## 2. ARQUITECTURA CLIENTE-SERVIDOR: DESVENTAJAS

- No hay un modelo de datos compartidos, así que los subsistemas utilizan diferentes datos de la organización. Intercambio de datos puede ser ineficaz;
- No hay registro central de nombres y servicios - que puede ser difícil de averiguar qué servidores y servicios están disponibles.
- Punto único de fallo: Si el servidor falla, todos los clientes se ven afectados.
- Sobrecarga del servidor: El servidor puede saturarse si recibe demasiadas solicitudes.

# Arquitectura de Microservicios

Divide el sistema en servicios pequeños, independientes y desplegados de manera individual. Cada microservicio se encarga de una funcionalidad específica y se comunica con otros servicios a través de protocolos de red ligeros (ej., HTTP, REST, gRPC).

# Arquitectura de Microservicios

## 3. ARQUITECTURA DE MICROSERVICIOS: VENTAJAS

- Escalabilidad independiente: Los microservicios pueden escalarse de manera individual.
- Flexibilidad tecnológica: Cada microservicio puede usar tecnologías diferentes.
- Despliegue ágil: Los microservicios se pueden desarrollar y desplegar de forma independiente.

# Arquitectura de Microservicios

## 3. ARQUITECTURA DE MICROSERVICIOS: DESVENTAJAS

Complejidad: Gestionar múltiples microservicios puede ser complicado.

Sobrecarga de red: La comunicación entre microservicios puede introducir latencia.

Dificultad en la depuración: El diagnóstico de errores es más difícil debido a la distribución.

# Arquitectura de Microservicios

## 3. AMAZON EMPLEA MICROSERVICIOS

De qué partían: En sus inicios, Amazon tenía una arquitectura monolítica, pero a medida que la compañía crecía, la complejidad y el tamaño del código aumentaron exponencialmente. Esto hacía difícil mantener y escalar la plataforma.

Cambio: Amazon migró su arquitectura a microservicios. Cada equipo era responsable de un "servicio" independiente y utilizaba APIs para la comunicación entre los servicios. Esto permitió un desarrollo más rápido y la habilidad de escalar partes específicas de la plataforma de manera independiente.

-----

Otras empresas como **Uber** y **Spotify** también migraron de una estructura inicial monolítica a microservicios cuando tuvieron la necesidad de crecimiento.

Impacto: La migración a microservicios facilitó la creación de Amazon Web Services (AWS), que es uno de los servicios de infraestructura en la nube más grandes del mundo. AWS proporciona a otros desarrolladores la capacidad de utilizar microservicios y escalabilidad de la misma manera que Amazon lo hacía internamente.

# Arquitectura de Capas N

Similar a la arquitectura en capas, pero con la distinción de que las capas se pueden distribuir físicamente en diferentes servidores o ubicaciones. Típicamente tiene capas como la capa de presentación, capa de aplicación (lógica de negocio) y capa de datos.

# Arquitectura de Capas N

## VENTAJAS

- Distribución física: Las capas pueden ejecutarse en diferentes servidores.
- Seguridad: Es más fácil controlar el acceso a los datos distribuyendo los niveles.

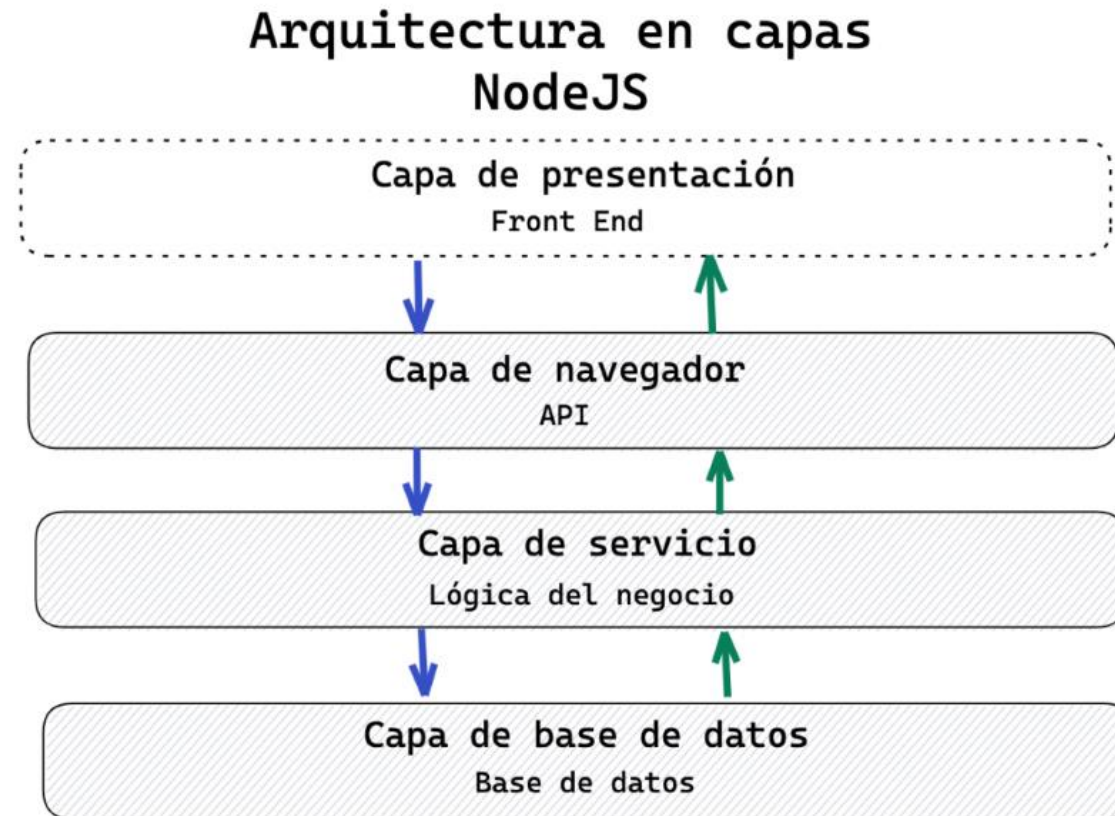
## DESVENTAJAS

- Complejidad: Añadir más capas introduce complejidad en el desarrollo y mantenimiento.
- Rendimiento: La interacción entre las capas distribuidas puede generar latencia.

## EJEMPLO

Aplicaciones empresariales distribuidas que manejan un alto volumen de datos y procesamiento.

# Arquitectura de Capas N: Ejemplo





# Arquitectura Orientada a Servicios

## 5. ARQUITECTURA ORIENTADA A SERVICIOS

Se basa en la creación de servicios reutilizables que ofrecen funcionalidad a través de interfaces bien definidas. Los servicios se comunican entre sí a través de un bus de servicio o directamente usando protocolos estándar como SOAP o REST.

### VENTAJAS

Interoperabilidad: Los servicios pueden comunicarse usando protocolos estándar, independientemente de la tecnología.

Reusabilidad: Los servicios pueden ser reutilizados en múltiples aplicaciones.

### DESVENTAJAS

Sobrecarga de comunicación: Los servicios se comunican a través de la red, lo que puede introducir latencia.

Complejidad de administración: Gestionar y coordinar muchos servicios puede ser complejo.

### EJEMPLO

Sistemas grandes como los de banca o telecomunicaciones que necesitan interoperabilidad entre diferentes servicios.

# Arquitectura orientada a servicios

## AIRBNB

- De qué partían: Airbnb utilizaba una arquitectura monolítica basada en Rails, lo que le sirvió en las primeras etapas de crecimiento. Sin embargo, a medida que la empresa escaló globalmente, el código monolítico se volvió más difícil de gestionar.
- Cambio: Airbnb migró a microservicios y, en algunos casos, también utilizó un enfoque "SOA" (Arquitectura Orientada a Servicios), lo que les permitió descomponer funcionalidades específicas, como la gestión de reservas, mensajes y pagos en servicios separados.
- Impacto: La migración a microservicios permitió a Airbnb mejorar la velocidad de desarrollo, escalar más eficientemente y manejar con mayor estabilidad la alta demanda en sus servicios.

# Arquitectura orientada a servicios

## EBAY

De qué partían: eBay comenzó como un sitio monolítico, y con el crecimiento del comercio electrónico, la plataforma se volvió cada vez más difícil de mantener. Las actualizaciones y cambios impactaban todo el sistema, y el tiempo de inactividad era frecuente durante los despliegues.

Cambio: En los años 2000, eBay migró a una arquitectura orientada a servicios (SOA), descomponiendo el monolito en servicios más pequeños y luego avanzando hacia microservicios.

Impacto: El cambio permitió a eBay escalar globalmente, mejorar la disponibilidad de su plataforma y reducir los tiempos de inactividad durante los despliegues.

# Arquitectura orientada a servicios

## EBAY

De qué partían: eBay comenzó como un sitio monolítico, y con el crecimiento del comercio electrónico, la plataforma se volvió cada vez más difícil de mantener. Las actualizaciones y cambios impactaban todo el sistema, y el tiempo de inactividad era frecuente durante los despliegues.

Cambio: En los años 2000, eBay migró a una arquitectura orientada a servicios (SOA), descomponiendo el monolito en servicios más pequeños y luego avanzando hacia microservicios.

Impacto: El cambio permitió a eBay escalar globalmente, mejorar la disponibilidad de su plataforma y reducir los tiempos de inactividad durante los despliegues.

# Ejercicio

- Explicar Arquitectura Monolítica
- Explicar Arquitectura de Microkernel o Plug-in
- Explicar Arquitectura Basada en componentes
- Explicar Arquitectura Basada en Nubes
- Explicar Modelo Vista Controlador
- Explicar Arquitectura Orientada a Objetos

# Antes de Concluir la Sesión

Explica el modelo arquitectónico que más te interesó.

# Tarea para el Viernes Próximo

Tabla comparativa de Modelos Arquitectónicos(5)

- Definición
- Es Jerárquica o no
- Ventajas
- Desventajas
- Ejemplos (2)
- Porqué lo usarías tú?

# Curso: Ingeniería de Software

## Ingeniería en Sistemas y Ciencias de la Computación

### Clase 8: Diseño UML

Catedrático: Claudia Alegre

7 de marzo de 2026



# Dinámica de Introducción

Identifiquemos un escenario :

## Una cafetería

### •Clientes y pedidos

- Los clientes llegan, hacen pedidos y pagan.
- Cada pedido puede incluir varios productos (café, pastel, jugo).

### •Menú y productos

- El menú agrupa los productos disponibles.
- Cada producto tiene nombre, precio y categoría (bebida, postre, snack).

### •Meseros y atención

- Los meseros toman pedidos y los entregan.
- Pueden atender a varios clientes a la vez.

### •Caja y pagos

- El cajero registra los pagos.
- Se pueden usar diferentes métodos de pago (efectivo, tarjeta).

### •Inventario

- La cafetería controla ingredientes (café, leche, azúcar).
- Los productos dependen de que haya inventario suficiente.

### •Ambiente social

- Los clientes pueden sentarse en mesas individuales o compartir mesa.
- Se generan interacciones: “cliente conversa con cliente”.

### ELEMENTOS:

- Clientes
- Cocinar
- Cafetera
- Orden personalizada
- Mobiliario
- Dueño
- Variedad de café
- Empleados
- Menu
- Facturación
- Inventario
- Gastos funcionamiento
- Pedidos
- Caja registradora
- Cajeros
- Cocineros
- Numeros mesa, organización local
- Baristas
- Seguridad
- Proveedores
- Ubicación
- Punto de venta
- Uniformes

# Tema

Diseño orientado a objetos con el uso del UML

# Indicadores de Logro

- Desarrolla diseños detallados que incluyan diagramas como diagramas de clases, secuencia y actividades para representar los componentes y la lógica del sistema.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Lenguaje Unificado de Modelado (UML)

UML es un lenguaje de modelado gráfico estandarizado utilizado para visualizar, especificar, construir y documentar los componentes de sistemas orientados a objetos.

Este lenguaje proporciona una forma común de modelar tanto el diseño estructural como el comportamiento de un sistema, lo que facilita la comunicación entre los diferentes actores del proceso de desarrollo (desarrolladores, analistas, clientes, etc.).

# Teoría: Lenguaje Unificado de Modelado (UML)

- Diagramas de casos de uso: representan a los actores y casos de uso (procesos principales) que intervienen en un desarrollo de software.
- Diagramas de clases: para UML una clase es una entidad, no una clase software. Un diagrama de clases UML puede ser un diagrama del dominio o representación de conceptos que intervienen en un problema, o también un diagrama de clases software. El sentido de un diagrama UML se lo da la persona que lo construye.
- Diagramas de secuencia: suelen usarse para representar objetos software y el intercambio de mensajes entre ellos, representando la aparición de nuevos objetos de izquierda a derecha.
- Diagramas de colaboración: suelen usarse para representar objetos o clases y la forma en que se transmiten mensajes y colaboran entre ellos para cumplir un objetivo.
- Diagramas de estados: suelen usarse para representar cómo evoluciona un sistema (cómo va cambiando de estado) a medida que se producen determinados eventos.
- Otros diagramas: etc. diagramas de actividad, diagramas de paquetes, diagramas de arquitectura software,

# Teoría:

## Lenguaje Unificado de Modelado (UML) - Sintaxis

UML utiliza una serie de elementos gráficos para representar los diferentes aspectos de un sistema.

**Clases:** Representadas por un rectángulo con tres divisiones (nombre, atributos, métodos).

**Relaciones:** Existen diferentes tipos de relaciones en UML, como la asociación, la herencia, la agregación, la composición y la dependencia. Se representan mediante diferentes tipos de líneas y flechas.

**Objetos:** Se representan con un rectángulo con subrayado y muestran una instancia de una clase específica.

**Componentes y Paquetes:** Representan módulos de software o subsistemas y se visualizan con formas rectangulares o como carpetas.

Teoría:

Lenguaje Unificado de Modelado (UML)

Ejemplo de representación de una clase

```
+-----+
|      Persona      |
+-----+
| + nombre: String  |
| + edad: int       |
+-----+
| + getNombre():String |
| + setEdad(edad: int): void |
+-----+
```



# Teoría: Lenguaje Unificado de Modelado (UML)

## Aspectos Conceptuales:

UML permite representar tanto la **estructura** del sistema (diagramas de clases, de objetos, de componentes) como su **comportamiento** (diagramas de secuencia, diagramas de actividades).

UML facilita el **análisis y diseño** en las etapas tempranas del desarrollo, permitiendo realizar ajustes y mejoras antes de la implementación.

## Aspectos Técnicos:

UML está diseñado para ser **independiente del lenguaje de programación**, lo que lo convierte en una herramienta flexible.

Se puede integrar con herramientas CASE (Computer-Aided Software Engineering) para generar código automáticamente a partir de los diagramas.

**Modelado orientado a objetos:** UML se basa en los principios de la programación orientada a objetos (clases, objetos, herencia, encapsulación, etc.).

# Teoría: Lenguaje Unificado de Modelado (UML)

## **Diagramas Estáticos (Estructurales)**

Estos diagramas modelan la estructura estática del sistema, es decir, la forma en que se organizan sus componentes.

- **Diagrama de clases:** Muestra la estructura del sistema mediante la representación de clases, atributos, métodos y las relaciones entre ellas.
- **Diagrama de objetos:** Representa instancias de clases en un momento específico, mostrando cómo los objetos se relacionan entre sí.
- **Diagrama de componentes:** Modela los módulos o componentes del sistema y sus relaciones.
- **Diagrama de paquetes:** Muestra cómo se agrupan las clases y otros elementos en paquetes o módulos lógicos.

# Teoría: Lenguaje Unificado de Modelado (UML)

## **Diagramas Dinámicos (De Comportamiento)**

Estos diagramas representan el comportamiento dinámico del sistema, es decir, cómo cambian los estados de los objetos o cómo se comunican entre sí a lo largo del tiempo.

- **Diagrama de secuencia:** Representa cómo los objetos interactúan en una secuencia temporal mediante el intercambio de mensajes.
- **Diagrama de actividades:** Modela el flujo de trabajo o las actividades del sistema, mostrando la lógica de los procesos.
- **Diagrama de casos de uso:** Muestra las interacciones entre los actores externos (usuarios u otros sistemas) y el sistema a través de diferentes casos de uso.
- **Diagrama de estado:** Representa los diferentes estados por los que pasa un objeto a lo largo de su ciclo de vida.

# Teoría: Lenguaje Unificado de Modelado (UML)

## Diagramas de Implementación

Estos diagramas se utilizan para modelar los aspectos físicos y de implementación de un sistema.

- **Diagrama de despliegue:** Muestra la arquitectura física del sistema, especificando cómo se distribuyen los componentes en hardware.
- **Diagrama de componentes:** Además de su uso estructural, puede mostrar cómo los componentes se implementan físicamente y cómo interactúan en tiempo de ejecución.

# Teoría: Lenguaje Unificado de Modelado (UML)

El diagrama de clases es un **diagrama puramente orientado al modelo de programación orientado a objetos**, ya que define las clases que se utilizarán cuando se pase a la fase de construcción y la manera en que se relacionan las mismas.

Se podría equiparar, al famoso diagrama de modelo Entidad-Relación (E/R), no recogido en UML, tiene una utilidad similar: la representación de datos y su interacción. Ambos diagramas muestran el modelo lógico de los datos de un sistema.

# Teoría: Lenguaje Unificado de Modelado (UML)

## Elementos de un diagrama de clases

El diagrama UML de clases está formado por dos elementos: clases, relaciones e interfaces.

### Clases

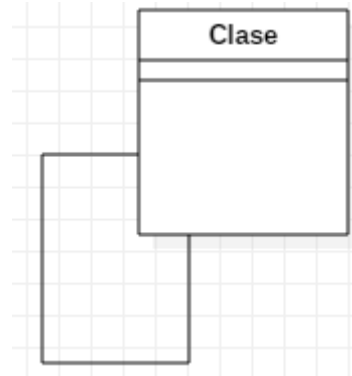
Las clases son el elemento principal del diagrama y representa, como su nombre indica, una clase dentro del paradigma de la orientación a objetos. Este tipo de elementos normalmente se utilizan para representar conceptos o entidades del «negocio». Una clase define un **grupo de objetos** que comparten características, condiciones y significado. La manera más rápida para encontrar clases sobre un enunciado, sobre una idea de negocio o, en general, sobre un tema concreto es buscar los **sustantivos** que aparecen en el mismo. Por poner algún ejemplo, algunas clases podrían ser: Animal, Persona, Mensaje, Expediente... Es un concepto muy amplio y **resulta fundamental identificar de forma efectiva estas clases**, en caso de no hacerlo correctamente se obtendrán una serie de problemas en etapas posteriores, teniendo que volver a hacer el análisis y perdiendo parte o todo el trabajo que se ha hecho hasta ese momento.

Bajando de nivel una clase está compuesta por tres elementos: **nombre de la clase, atributos, funciones. Estos elementos se incluyen en la representación** (o no, dependiendo del nivel de análisis).

# Teoría: Lenguaje Unificado de Modelado (UML)

## Relaciones

Una relación **identifica una dependencia**. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacia sí misma (menos común, pero existen), este último tipo de dependencia se denomina *dependencia reflexiva*. Las relaciones se representan con una línea que une las clases, esta línea variará dependiendo del tipo de relación

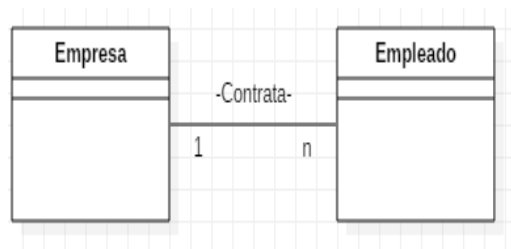


Relación reflexiva

# Teoría: Lenguaje Unificado de Modelado (UML)

Las relaciones en el diagrama de clases tienen varias propiedades, que dependiendo la profundidad que se quiera dar al diagrama se representarán o no. Estas propiedades son las siguientes:

- **Multiplicidad.** Es decir, el número de elementos de una clase que participan en una relación. Se puede indicar un número, un rango... Se utiliza  $n$  o  $*$  para identificar un número cualquiera.
- **Nombre de la asociación.** En ocasiones se escriba una indicación de la asociación que ayuda a entender la relación que tienen dos clases. Suelen utilizarse verbos como por ejemplo: «Una empresa contrata a  $n$  empleados»



Ejemplo de relación empleado-empresa



# Teoría: Lenguaje Unificado de Modelado (UML)

Tipos de relaciones

Un diagrama de clases incluye los siguientes tipos de relaciones:

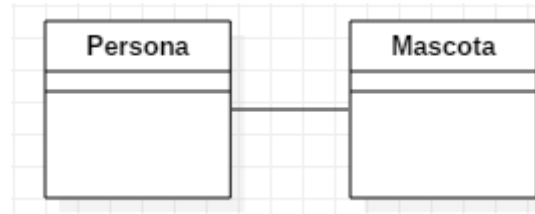
**Asociación.**

**Agregación.**

**Composición.**

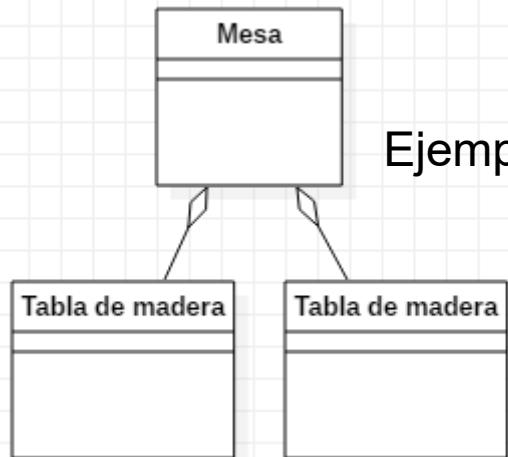
**Dependencia.**

**Herencia.**



Ejemplo de asociación

Este tipo de relación es el más común y se utiliza para representar dependencia semántica.



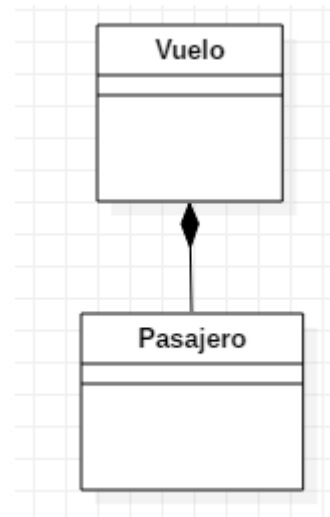
Ejemplo de Agregación

Es una representación jerárquica que indica a un objeto y las partes que componen ese objeto. Es decir, representa relaciones en las que **un objeto es parte de otro**, pero aun así debe tener **existencia en sí mismo**.

# Teoría: Lenguaje Unificado de Modelado (UML)

## Composición

La composición es similar a la agregación, representa una **relación jerárquica entre un objeto y las partes que lo componen, pero de una forma más fuerte**. En este caso, los elementos que forman parte no tienen sentido de existencia cuando el primero no existe. Es decir, cuando el elemento que contiene los otros desaparece, deben desaparecer todos ya que no tienen sentido por sí mismos sino que dependen del elemento que componen. Además, suelen tener los mismos tiempo de vida. Los componentes no se comparten entre varios elementos, esta es otra de las diferencias con la agregación.

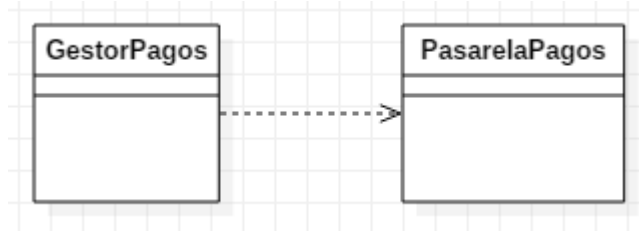


# Teoría: Lenguaje Unificado de Modelado (UML)

## Dependencia

Se utiliza este tipo de relación para **representar que una clase requiere de otra para ofrecer sus funcionalidades**. Es muy sencilla y se representa con una flecha discontinua que va desde la clase que necesita la utilidad de la otra flecha hasta esta misma.

Un ejemplo de esta relación podría ser la siguiente:

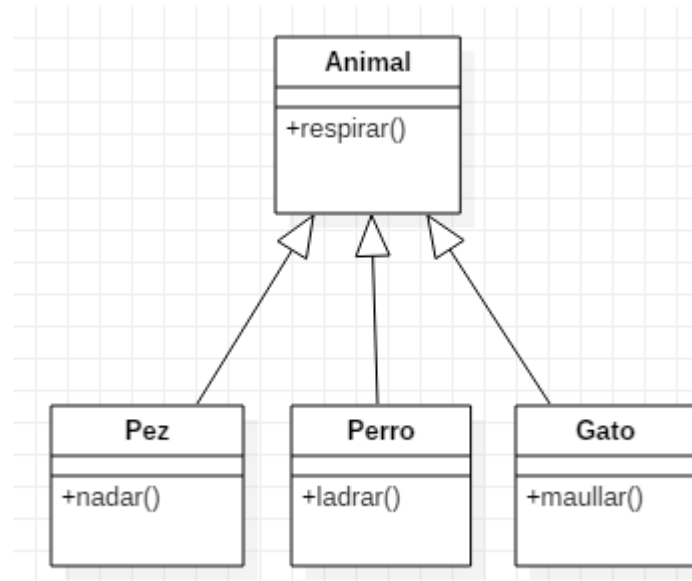


# Teoría: Lenguaje Unificado de Modelado (UML)

## Herencia

Otra relación muy común en el diagrama de clases es la herencia. Este tipo de relaciones permiten que **una clase (clase hija o subclase) reciba los atributos y métodos de otra clase (clase padre o superclase)**. Estos atributos y métodos recibidos se suman a los que la clase tiene por sí misma. Se utiliza en relaciones «es un».

Un ejemplo de esta relación podría ser la siguiente: Un pez, un perro y un gato son animales.



# Teoría: Lenguaje Unificado de Modelado (UML)

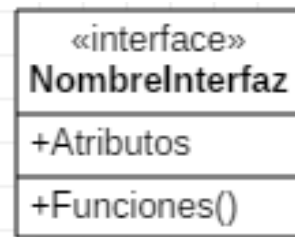
## Interfaces

Una interfaz es una entidad que declara una **serie de atributos, funciones y obligaciones**. Es una especie de contrato donde toda instancia asociada a una interfaz debe de implementar los servicios que indica aquella interfaz.

Dado que únicamente son declaraciones **no pueden ser instanciadas**.

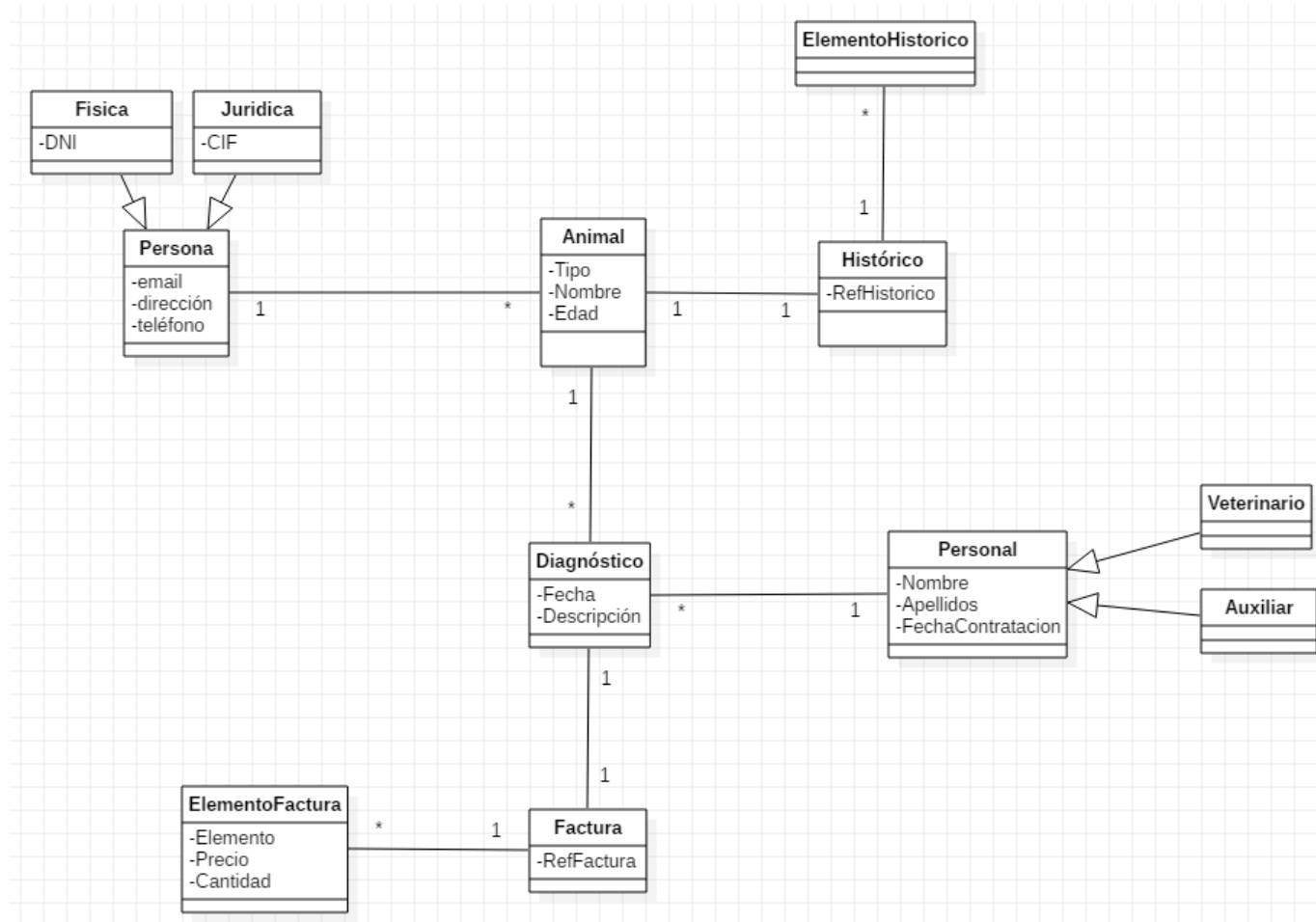
Las interfaces se asocian a clases. Una asociación entre una clase y una interfaz representa que esa clase cumple con el contrato que indica la interfaz, es decir, incluye aquellas funciones y atributos que indica la interfaz.

Su representación es similar a las clases, pero indicando arriba la palabra <<interface>>.



# Teoría: Diagrama de clases

Clínica veterinaria



# Identifiquemos clases en el ejemplo de la cafetería

- ....

# Identifiquemos clases en el ejemplo de la cafetería

## **Cliente**

- Atributos: nombre, formaPago
- Métodos: hacerPedido(), pagar()

## **Pedido**

- Atributos: numeroPedido, fecha
- Métodos: agregarProducto(), calcularTotal()

## **Producto**

- Atributos: nombre, precio, tipo (bebida, postre, snack)
- Métodos: mostrarInfo()

## **Mesero**

- Atributos: nombre, turno
- Métodos: tomarPedido(), entregarPedido()

## **Caja**

- Atributos: saldoDiario
- Métodos: registrarPago(), emitirFactura()

## **Inventario**

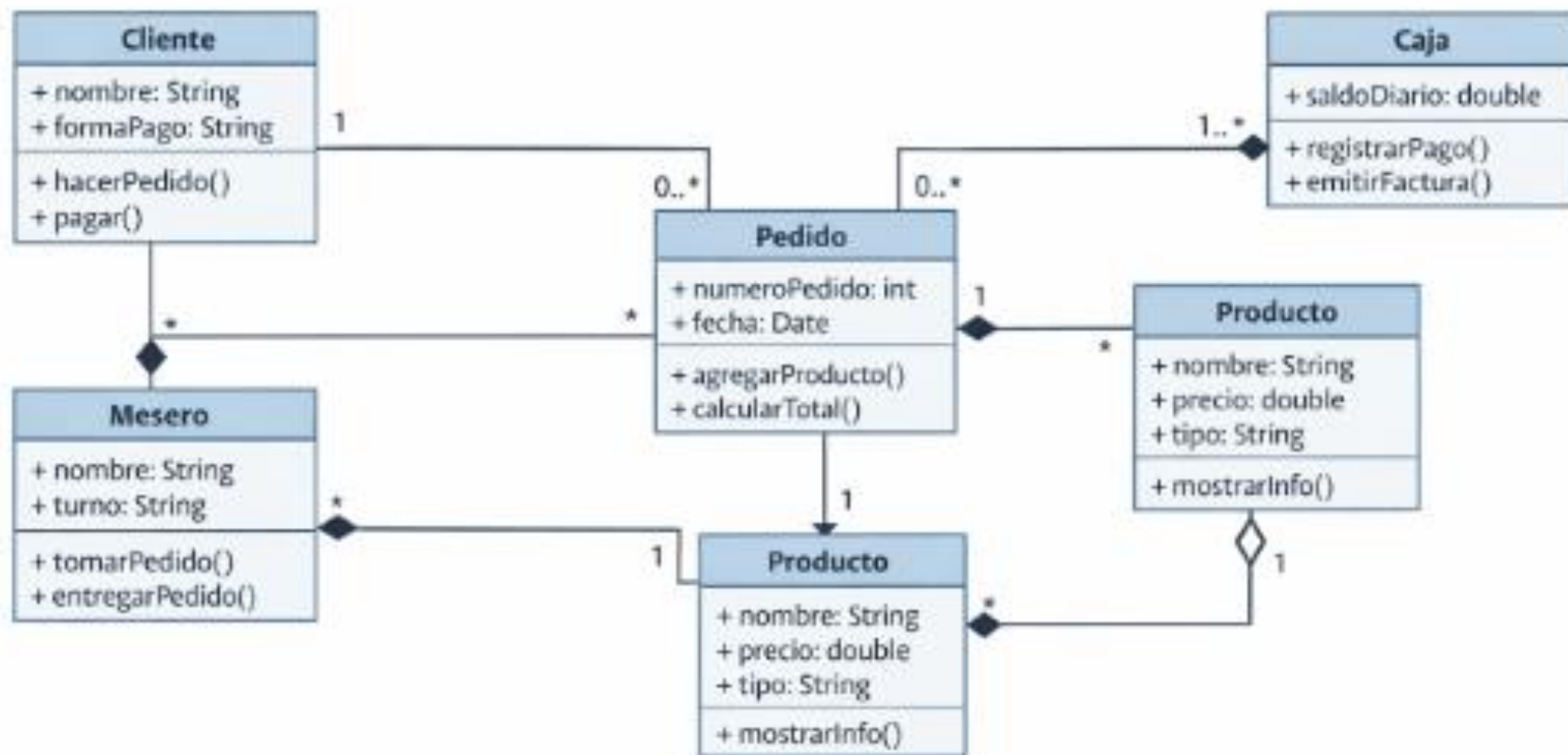
- Atributos: ingredientesDisponibles
- Métodos: actualizarStock(), verificarDisponibilidad()



# Relaciones

- **Cliente** ↔ **Pedido**: un cliente hace uno o varios pedidos.
- **Pedido** ↔ **Producto**: un pedido contiene varios productos.
- **Mesero** ↔ **Pedido**: el mesero atiende y gestiona pedidos.
- **Pedido** ↔ **Caja**: cada pedido se paga en la caja.
- **Producto** ↔ **Inventario**: los productos dependen de los ingredientes disponibles.

# DIAGRAMA



# Diagrama de actividades

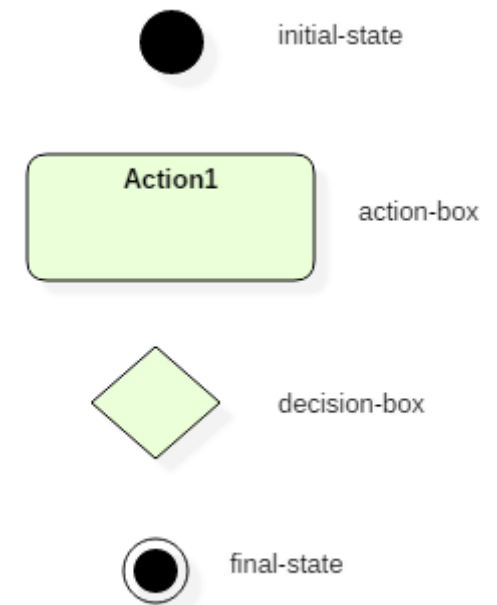
Es básicamente un diagrama de flujo para representar el flujo de una actividad a otra. La actividad puede describirse como una operación del sistema. El propósito básico de los diagramas de actividades es capturar el comportamiento dinámico del sistema. También se le llama diagrama de flujo orientado a objetos.

Un diagrama de actividades se utiliza para diagramas de actividades que describen el flujo de ejecución entre múltiples actividades

# Diagrama de actividades

## Notaciones del diagrama de actividades

- Los símbolos de diagramas de actividades se pueden generar utilizando las siguientes notaciones:
- Estados iniciales: la etapa inicial antes de que se lleve a cabo una actividad se describe como el estado inicial.
- Estados finales: El estado al que llega el sistema cuando finaliza un proceso específico se conoce como Estado Final.
- Estado o casilla de actividad:
- Cuadro de decisión: Es un cuadro en forma de diamante que representa una decisión con caminos alternativos. Representa el flujo de control.



# Diagrama de actividades con actores



| Aspecto              | Diagrama de Flujo                                                                 | Diagrama de Actividades (UML)                                                               |
|----------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Origen               | Tradicional en programación y procesos administrativos                            | Parte del Lenguaje Unificado de Modelado (UML)                                              |
| Propósito            | Representar algoritmos, pasos secuenciales y decisiones                           | Modelar procesos de negocio, flujos de trabajo y comportamiento dinámico de sistemas        |
| Nivel de detalle     | General, centrado en pasos y decisiones                                           | Más rico: incluye roles, objetos, concurrencia y sincronización                             |
| Símbolos principales | Óvalos (inicio/fin), rectángulos (procesos), rombos (decisiones), flechas (flujo) | Nodos de actividad, carriles de natación (roles), bifurcaciones, uniones, objetos, mensajes |
| Ámbito de uso        | Programación, documentación de procesos simples                                   | Ingeniería de software, gestión de proyectos, modelado de sistemas complejos                |
| Concurrencia         | No la representa                                                                  | Sí, permite modelar actividades paralelas                                                   |
| Roles/Responsables   | No distingue responsables                                                         | Carriles de natación muestran qué actor realiza cada actividad                              |

# Ejercicio

- Diagrama de actividades para proceso de compra en línea

El proceso de compra en línea comienza cuando el cliente accede a la tienda virtual y navega por el catálogo o utiliza el buscador para localizar un producto. Una vez que encuentra lo que desea, selecciona el artículo y lo agrega al carrito de compras.

En ese momento se presenta una decisión: continuar comprando o proceder al pago. Si el cliente decide seguir comprando, regresa al catálogo y repite la búsqueda y selección de productos; si opta por pagar, avanza al ingreso de los datos de pago.

El sistema valida la transacción y, si es correcta, confirma la compra y envía un comprobante al cliente por correo electrónico o notificación. En caso de que la transacción falle, se muestra un mensaje de error y el cliente debe volver a ingresar sus datos de pago. El proceso concluye con la confirmación de la compra y el envío del comprobante, marcando el fin del flujo.

# Diagrama de secuencia

Su objetivo es representar el intercambio de mensajes entre los distintos objetos del sistema para cumplir con una funcionalidad. Define, por tanto, el comportamiento dinámico del sistema de información.

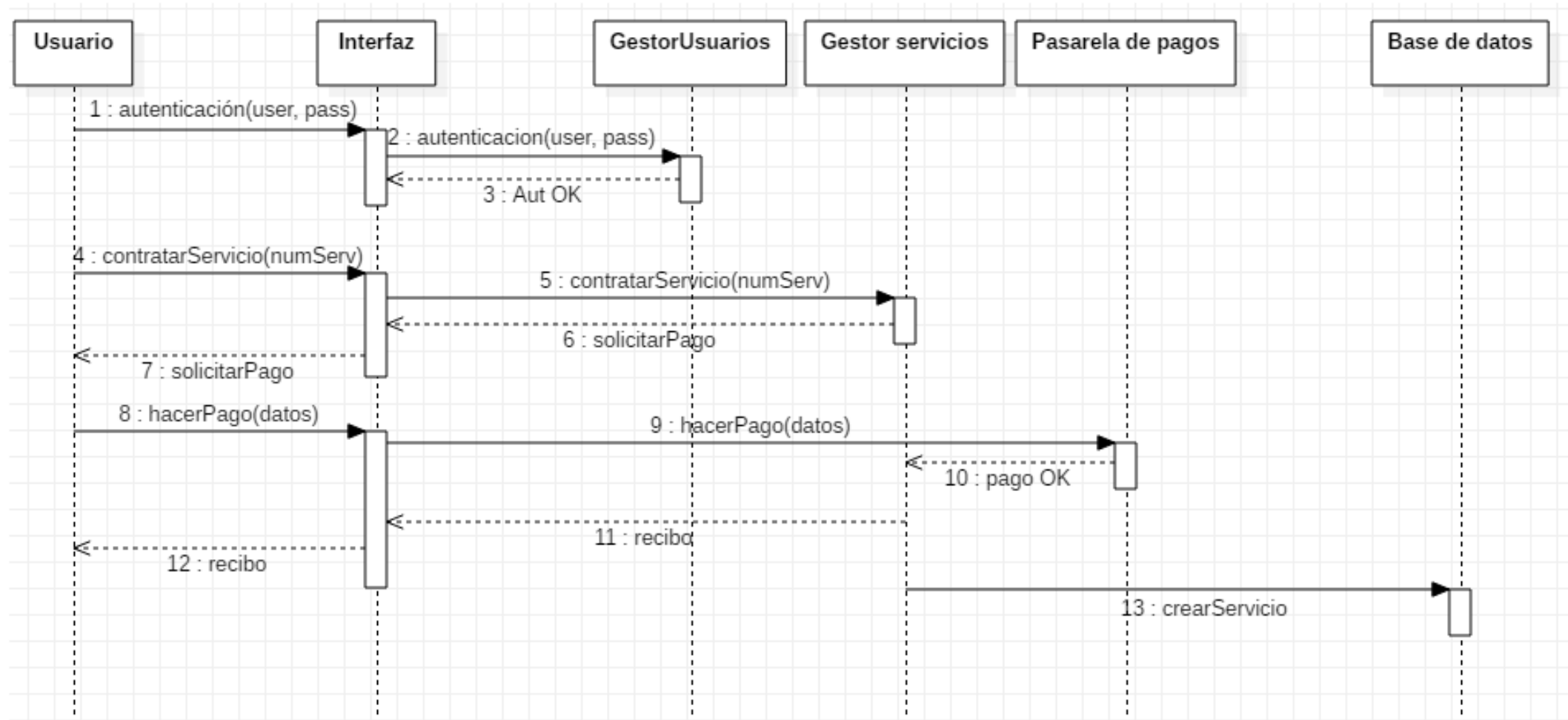
Normalmente es utilizado para definir como se realiza un caso de uso por lo que es comúnmente utilizado junto al diagrama de casos de uso. También se suele construir para comprender mejor el diagrama de clases, ya que el diagrama de secuencia muestra como objetos de esas clases interactúan haciendo intercambio de mensajes.



# Diagrama de secuencia

- El diagrama de secuencia está construido a partir de dos dimensiones:
- Horizontal: Representa los objetos que participan en la secuencia.
- Vertical: Representa la línea de tiempo sobre la que los elementos actúan. Va de arriba (menor tiempo) hacia abajo (menor tiempo) No es común «reglar» esta dimensión mediante una escala para poner tiempos específicos, a excepción de sistemas de tiempo real donde la velocidad para llevar a cabo la funcionalidad sí es relevante.

# Diagrama de secuencia



# Ejercicio

- Diagrama de secuencia para proceso de compra en línea

El proceso de compra en línea comienza cuando el cliente accede a la tienda virtual y navega por el catálogo o utiliza el buscador para localizar un producto. Una vez que encuentra lo que desea, selecciona el artículo y lo agrega al carrito de compras.

En ese momento se presenta una decisión: continuar comprando o proceder al pago. Si el cliente decide seguir comprando, regresa al catálogo y repite la búsqueda y selección de productos; si opta por pagar, avanza al ingreso de los datos de pago.

El sistema valida la transacción y, si es correcta, confirma la compra y envía un comprobante al cliente por correo electrónico o notificación. En caso de que la transacción falle, se muestra un mensaje de error y el cliente debe volver a ingresar sus datos de pago. El proceso concluye con la confirmación de la compra y el envío del comprobante, marcando el fin del flujo.

# Diagrama de secuencia

Hagamos un diagrama de secuencia para la  
cafetería

# Antes de Concluir la Sesión

Comparte tus modelos

# Tarea para el Viernes Próximo

Tabla comparativa de diagramas UML (10)

- Propósito
- Elementos clave
- Tipo de modelado
- Quien es el usuario de este diagrama
- Porqué lo usarías tú?

Recomiéndame 3 para un diseño de sistema de registro para USPG, Porqué utilizar cada uno de ellos.



# Curso: Ingeniería de Software

## Ingeniería en Sistemas y Ciencias de la Computación

### Clase 9: Pruebas de Software

Catedrático: Claudia Alegre

14 de marzo de 2026

# Dinámica de Introducción

Validaciones....

Validaciones....



# Dinámica de Introducción

Validaciones....

```
Algoritmo SumaPares
  Definir i, suma Como Entero
  suma ← 0
  Para i ← 1 Hasta 10 Hacer
    Si (i MOD 2 = 1) Entonces
      suma ← suma + i
    FinSi
  FinPara
  Escribir "La suma de los números pares del 1 al 10 es: ",
suma
FinAlgoritmo
```

```
Algoritmo SumaPares
  Definir i, suma Como Entero
  suma ← 0
  Para i ← 1 Hasta 10 Hacer
    Si (i MOD 2 = 0) Entonces
      suma ← suma + i
    FinSi
  FinPara
  Escribir "La suma de los números pares del 1 al 10 es: ",
suma
FinAlgoritmo
```

# Dinámica de Introducción

Validaciones....

```
Algoritmo ConvertirCelsiusAFahrenheit
  Definir celsius, fahrenheit Como Real
  Leer fahrenheit
  fahrenheit  $\leftarrow$  (celsius * 5/9) + 32
  Escribir "La temperatura en Fahrenheit es: ", fahrenheit
FinAlgoritmo
```

```
Algoritmo ConvertirCelsiusAFahrenheit
  Definir celsius, fahrenheit Como Real
  Leer celsius
  fahrenheit  $\leftarrow$  (celsius * 9/5) + 32
  Escribir "La temperatura en Fahrenheit es: ", fahrenheit
FinAlgoritmo
```

# Dinámica de Introducción

Validaciones....

```
Algoritmo Promedio
  Definir nota1, nota2, nota3, promedio Como Real
  Leer nota1
  Leer nota2
  promedio ← (nota1 + nota2 + nota3) / 2
  Escribir "El promedio es: ", promedio
FinAlgoritmo
```

```
Algoritmo Promedio
  Definir nota1, nota2, nota3, promedio Como Real
  Leer nota1
  Leer nota2
  Leer nota3
  promedio ← (nota1 + nota2 + nota3) / 3
  Escribir "El promedio es: ", promedio
FinAlgoritmo
```

# Tema

- Pruebas de desarrollo
- Pruebas de unidad
- Elección de casos de pruebas de unidad
- Pruebas de componentes
- Pruebas del sistema
- Desarrollo dirigido por pruebas
- Pruebas de versión
- Pruebas basadas en requerimientos
- Pruebas de escenario
- Pruebas de rendimiento
- Pruebas de usuario

# Indicadores de Logro

- Explica la importancia de las pruebas de software en el ciclo de desarrollo y su rol en garantizar la calidad del producto.
- Diferencia entre los principales tipos de pruebas: unitarias, de integración, funcionales, de aceptación y de sistema.
- Analiza los resultados de las pruebas para verificar que el software cumple con los criterios de aceptación definidos.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Pruebas de software

## **¿Qué son las pruebas de software?**

Las pruebas de software son el proceso de evaluar una aplicación de software para asegurar que su calidad, funcionalidad y rendimiento se alinean con las expectativas y requisitos del usuario.

Para realizar pruebas, los testers ejecutan el software bajo condiciones controladas, simulando una variedad de escenarios, entornos e interacciones con el usuario. Este enfoque les permite identificar cualquier defecto que pueda surgir durante el proceso de prueba.

# Teoría: Pruebas de software

## ¿Por qué son cruciales las pruebas de software?

Algunos de los beneficios de las pruebas de software incluyen:

- Detección y prevención de errores: identifica y resuelve errores o problemas (fallos arquitectónicos, mal diseño, funcionalidad inválida, problemas de seguridad y escalabilidad) al principio del proceso de desarrollo, reduciendo costes y tiempo
- Mejorar la calidad del producto: garantiza que el software funcione de forma eficiente en diversas plataformas, dispositivos y entornos
- Mayor confianza del cliente: Un software de alta calidad conduce a una experiencia digital sin fricciones y a clientes satisfechos
- Requisitos de cumplimiento: Para sectores donde el cumplimiento normativo es crucial (como sanidad, finanzas o aviación), es fundamental cumplir con las normativas y estándares regulatorios. Las pruebas ayudan a garantizar que el software cumpla con las normativas del sector.



# Teoría: Pruebas de software

## **Pruebas funcionales**

Este tipo de pruebas se centra en verificar si la aplicación se comporta según los requisitos especificados. Comprueba las funciones del sistema alimentándoles entradas y examinando las salidas para asegurarse de que coinciden con los resultados esperados. El objetivo principal de las pruebas funcionales es validar que cada característica del software funciona correctamente

Este término general incluye una variedad de tipos de pruebas, cada una diseñada para cumplir un propósito específico.

# Teoría: Pruebas de software

## Pruebas no funcionales

Por otro lado, este tipo de prueba evalúa el rendimiento, la usabilidad, la fiabilidad y otros atributos de una aplicación de software que no se relacionan directamente con funcionalidades específicas

Bajo las pruebas no funcionales, existen muchos tipos de pruebas:

- **Pruebas de seguridad:** evalúan la capacidad del software para proteger frente a amenazas y vulnerabilidades
- **Pruebas de rendimiento:** evalúan la velocidad, capacidad de respuesta y estabilidad del sistema bajo diferentes condiciones.
- **Pruebas de usabilidad:** mide lo fácil e intuitivo que es el software para los usuarios
- **Pruebas de compatibilidad:** verifica que el software funciona en diversas plataformas, dispositivos, sistemas operativos y navegadores.

# Teoría: Pruebas de software

¿Qué es más caro que las pruebas de software? Lanzar un producto lleno de defectos.

Un sólido plan de pruebas de software ayuda a evitar costosos fallos posteriores al lanzamiento asegurando pruebas estructuradas, eficientes y completas. Minimiza riesgos, optimiza los recursos y alinea a tu equipo hacia un desarrollo orientado a la calidad.

Tanto si eres una startup como una empresa, tener una hoja de ruta clara para las pruebas significa un tiempo de lanzamiento al mercado más rápido, menos quejas de clientes y menores costes de mantenimiento. Vamos a explorar cómo diseñar un plan de pruebas de software que ahorre tiempo, reduzca costes y asegure un lanzamiento de productos fluido

# Teoría: Pruebas de software

Un plan de pruebas bien estructurado consta de varios componentes cruciales:

- **Alcance:** Define los objetivos del proyecto y describe los escenarios de usuario que se van a probar. También puede indicar cualquier escenario o problema que se excluya intencionadamente.
- **Horario:** Establece las fechas de inicio y los plazos para que los evaluadores completen sus evaluaciones y entreguen los resultados.
- **Asignación de recursos:** Asigna tareas específicas de prueba a los evaluadores designados, asegurando claridad en las responsabilidades.
- **Entorno de prueba:** Describe la configuración, configuración y accesibilidad del entorno de pruebas.
- **Herramientas de prueba:** Enumera las herramientas que se utilizarán para ejecutar pruebas, rastrear errores y gestionar otras actividades relacionadas con las pruebas.
- **Gestión de defectos:** Detalla el proceso para reportar problemas, especificando quién debe recibir los informes de errores y qué materiales de apoyo (como capturas de pantalla, registros o vídeos) deben incluirse.
- **Gestión de riesgos:** Identifica los riesgos potenciales que pueden surgir durante las pruebas y destaca los riesgos asociados a la publicación del software sin una evaluación exhaustiva.
- **Criterios de salida:** Define las condiciones bajo las cuales concluirán las pruebas, proporcionando resultados esperados que los testers pueden utilizar para evaluar la calidad global del software.

# Teoría: Pruebas de software

| Componente            | Detalles                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Objetivos             | <p>El objetivo es garantizar que las funcionalidades principales de la aplicación bancaria (gestión de cuentas, transferencias de fondos, pagos de facturas y medidas de seguridad) funcionen según lo esperado, cumpliendo con los estándares de rendimiento, usabilidad y cumplimiento de seguridad. Los objetivos específicos incluyen:</p> <ul style="list-style-type: none"><li>•Verificar el procesamiento de transacciones sin interrupciones</li><li>•Garantizar medidas de seguridad de alto nivel (autenticación multifactor, cifrado)</li><li>•Prueba compatibilidad entre dispositivos (Android/iOS)</li></ul> |
| Enfoque               | <p>La estrategia de pruebas combina pruebas manuales y automatizadas.</p> <ul style="list-style-type: none"><li>•Las pruebas manuales cubrirán la usabilidad, exploratoria y pruebas de seguridad</li><li>•Las pruebas automatizadas gestionarán las pruebas de regresión, rendimiento y carga</li><li>•Se priorizarán las pruebas funcionales, de rendimiento y de seguridad</li></ul>                                                                                                                                                                                                                                    |
| Alcance               | <p>En alcance</p> <ul style="list-style-type: none"><li>•Autenticación de usuario (biométrica, OTP, PIN)</li><li>•Transferencias de fondos (internas y externas)</li><li>•Pagos de facturas y recargas de móviles</li><li>•Historial de transacciones y extractos</li><li>•Notificaciones push para transacciones</li></ul>                                                                                                                                                                                                                                                                                                |
|                       | <p>Fuera de alcance</p> <ul style="list-style-type: none"><li>•Versiones heredadas del sistema operativo por debajo de Android 7/iOS 13</li></ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Entregables de prueba | <ul style="list-style-type: none"><li>•Casos de prueba (automatizados y manuales)</li><li>•Informes de defectos</li><li>•Informes de ejecución de pruebas</li><li>•Informe final de la prueba</li></ul> <p>Estos entregables se compartirán quincenalmente con las partes interesadas y se archivarán para futuras pruebas de regresión.</p>                                                                                                                                                                                                                                                                               |

| Componente                | Detalles                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dependencias              | <ul style="list-style-type: none"><li>•Integración con pasarelas de pago de terceros</li><li>•Cumplimiento de la normativa financiera (PCI DSS, RGPD)</li><li>•Dispositivos móviles actualizados (Android 14, iOS 17)</li></ul> <p>Plan de contingencia: Utilizar servicios simulados para dependencias de la API si no están disponibles</p>                                                                                                                                                               |
| Entorno de pruebas        | <ul style="list-style-type: none"><li>•Dispositivos: Android 11-14, iOS 14-17</li><li>•Servidor: Linux con PostgreSQL</li><li>•Red: Datos móviles (4G/5G) y pruebas Wi-Fi</li></ul>                                                                                                                                                                                                                                                                                                                         |
| Gestión de riesgos        | <p>Riesgos identificados:</p> <ul style="list-style-type: none"><li>•El alto tráfico provoca sobrecarga del sistema</li><li>•Fallos de API que afectan al procesamiento de transacciones</li><li>•Vulnerabilidades de seguridad en los módulos de autenticación</li></ul> <p>Estrategias de mitigación:</p> <ul style="list-style-type: none"><li>•Realizar pruebas de carga rigurosas</li><li>•Implementar mecanismos de respaldo de la API</li><li>•Realizar pruebas de penetración exhaustivas</li></ul> |
| Calendario                | <p>Cronología:</p> <ul style="list-style-type: none"><li>•Planificación de la prueba: 1 – 3 de abril</li><li>•Diseño de casos de prueba: 4 de abril – 6 de abril</li><li>•Pruebas funcionales: 7 de abril – 9 de abril</li><li>•Pruebas de seguridad y rendimiento: 10 de abril – 12 de abril</li><li>•Informes de pruebas: 13 de abril – 14 de abril</li></ul>                                                                                                                                             |
| Roles y responsabilidades | <ul style="list-style-type: none"><li>•Responsable de QA: John Doe – Responsabilidad general de la ejecución y la presentación de informes de pruebas</li><li>•Ingenieros de prueba: Ejecutan casos de prueba e informan de defectos</li><li>•Desarrolladores: Soluciona los problemas identificados durante las pruebas</li><li>•DevOps: Configurar y mantener el entorno de pruebas</li></ul>                                                                                                             |

# Teoría: Pruebas de software

**Las pruebas unitarias** se centran en verificar la funcionalidad de unidades o componentes individuales de forma aislada.

**Las pruebas de integración** aseguran que estas piezas separadas funcionen perfectamente en un entorno combinado.

# Teoría: Pruebas de software

## **Características de las pruebas unitarias**

**Aislamiento:** Las pruebas se centran en una sola unidad de código, con dependencias normalmente simuladas o eliminadas.

**Ejecución rápida:** Como prueban componentes pequeños, las pruebas unitarias suelen ejecutarse rápidamente y pueden activarse con frecuencia.

**Retroalimentación inmediata:** Los equipos identifican los problemas a tiempo, lo que facilita y abarata la corrección.

**Alto volumen:** Una base de código robusta puede incluir cientos o miles de pruebas unitarias.

# Teoría: Pruebas de software

## **Ventajas de las pruebas unitarias**

**Detección temprana de errores:** Identificar defectos a nivel de componente evita errores mayores y complejos más adelante.

**Mejora de la calidad del código:** Escribir pruebas simultáneamente con las características fomenta un mejor diseño y mantenimiento.

**Facilita la refactorización:** Cambiar la lógica de una función es menos arriesgado cuando existen pruebas unitarias completas, ya que revelan rápidamente regresiones.



# Teoría: Pruebas de software

## **Características de las pruebas de integración**

**Dependencias reales:** A diferencia de las pruebas unitarias, las pruebas de integración suelen emplear bases de datos reales, APIs y servicios externos.

**Entornos complejos:** Una configuración integrada puede involucrar múltiples componentes, lo que requiere entornos de prueba bien orquestados.

**Cobertura más amplia:** Prueba múltiples interacciones, ofreciendo una visión más holística de cómo funciona el sistema.

**Tiempo de ejecución moderado a alto:** Debido a la complejidad de la configuración y las llamadas de servicio reales, las pruebas de integración generalmente duran más que las pruebas unitarias.

Plan detallado de Pruebas Unitarias / Casos de Uso

Sistema:  
Módulo:  
Fecha de elaboración:  
Fecha planeada de ejecución:

Para capacitación y pruebas unitarias

| Para capacitación y pruebas unitarias |  |                                                                                         |             |                 |                   | Probado |    |                    | Pasa la prueba |    | Descripción de prueba             |
|---------------------------------------|--|-----------------------------------------------------------------------------------------|-------------|-----------------|-------------------|---------|----|--------------------|----------------|----|-----------------------------------|
| AREA                                  |  | LISTA DE PROCESOS / CASOS DE USO                                                        | RESPONSABLE | Fecha de prueba | Datos para prueba | SI      | NO | Resultado Esperado | SI             | NO | Error / comentarios/observaciones |
| INVENTARIOS                           |  | GESTION DE INVENTARIOS                                                                  |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Procedimiento para gestionar ingreso de produccion a enfriamiento                       |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Procedimiento para gestionar traslados entre bodegas                                    |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Gestión de ingresos de Pedidos de Venta                                                 |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Productos terminados                                                                    |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Consulta de Existencias de PTS                                                          |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Orden de produccion como ingreso y transferencias como salidas                          |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Recepción de pedidos de compra para productos de terceros                               |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Despacho por ventas locales y exportación                                               |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Generacion de Disponible, descargar a excel para tener listado de conteo de inventario. |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | En la opcion de diarios de recuento se realizan los ajustes de inventario.              |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Traslados de Materiales                                                                 |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Informe de Inventario Disponible de Mezclas                                             |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Rechazos de Produccion                                                                  |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Recuentos(Ajustes de Invetarios Materia Prima)                                          |             |                 |                   |         |    |                    |                |    |                                   |
| INVENTARIOS                           |  | Cobertura Mínimos y Máximos                                                             |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | GESTION DE DESPACHO                                                                     |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Registro de Traslados de M.E.                                                           |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Consulta de Existencis de PTS                                                           |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Gestión de Despacho de Pedido Locales                                                   |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Gestión de Despacho de Exportaciones                                                    |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Packing List de Pedidos                                                                 |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Creación y registro de Itinerarios y Rutas de despacho                                  |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Informes y Consulas de Inventarios Generales                                            |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Consulta de proveedores                                                                 |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Lista de Empaque de Pedidos                                                             |             |                 |                   |         |    |                    |                |    |                                   |
| LOGISTICA                             |  | Consulta de listado de Clientes y datos generales                                       |             |                 |                   |         |    |                    |                |    |                                   |

Total casos de uso planificados

Total casos de uso probados

% de cobertura

# Ejercicio

## **Caso: Sistema de Reservas de Consultas Médicas en Línea**

Una clínica privada ha implementado un sistema web para que los pacientes reserven citas médicas.

El sistema debe permitir:

Registro de pacientes nuevos. Inicio de sesión de pacientes existentes. Búsqueda de médicos por especialidad. Selección de fecha y hora disponibles. Confirmación de la cita con envío de correo electrónico.

### **Registro de paciente**

- El correo electrónico debe ser único.
- La contraseña debe tener al menos 8 caracteres, incluir una mayúscula y un número.
- El número de teléfono debe tener exactamente 8 dígitos.

### **Reserva de citas:**

- Un paciente no puede reservar dos citas en la misma franja horaria.
- El sistema solo muestra horarios disponibles según la agenda del médico.
- La confirmación genera un código único de reserva.

### **Cancelación de citas:**

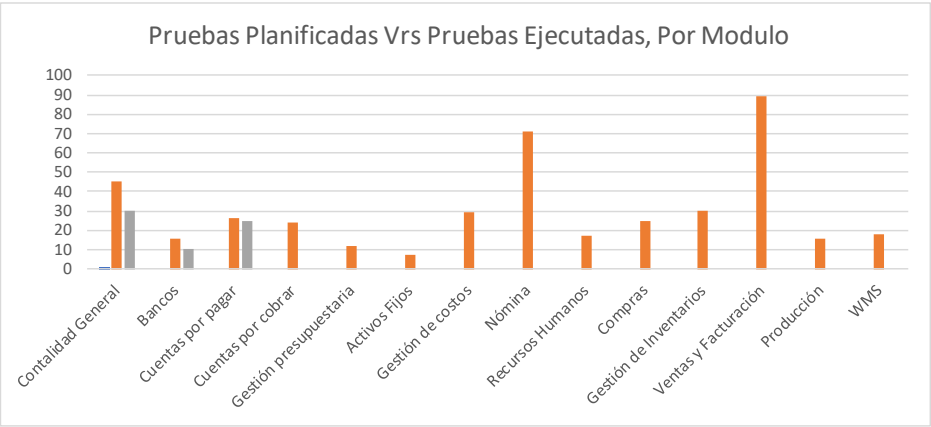
- Solo se puede cancelar hasta 24 horas antes de la cita.
- El sistema debe enviar un correo de confirmación de cancelación.

Elabora un plan de pruebas unitarias, con los temas vistos en clase.

Resumen General de avance de pruebas unitarias

Actualizado al:

| Módulo                 | Escenarios de prueba planificados | Escenarios ejecutados | %   |
|------------------------|-----------------------------------|-----------------------|-----|
| Contalidad General     | 45                                | 30                    | 67% |
| Bancos                 | 16                                | 10                    | 63% |
| Cuentas por pagar      | 26                                | 25                    | 96% |
| Cuentas por cobrar     | 24                                |                       | 0%  |
| Gestión presupuestaria | 12                                |                       | 0%  |
| Activos Fijos          | 7                                 |                       | 0%  |
| Gestión de costos      | 29                                |                       | 0%  |
| Nómina                 | 71                                |                       | 0%  |
| Recursos Humanos       | 17                                |                       | 0%  |
| Compras                | 25                                |                       | 0%  |
| Gestión de Inventarios | 30                                |                       | 0%  |
| Ventas y Facturación   | 89                                |                       | 0%  |
| Producción             | 16                                |                       | 0%  |
| WMS                    | 18                                |                       | 0%  |
| Total                  | 425                               | 65                    | 15% |



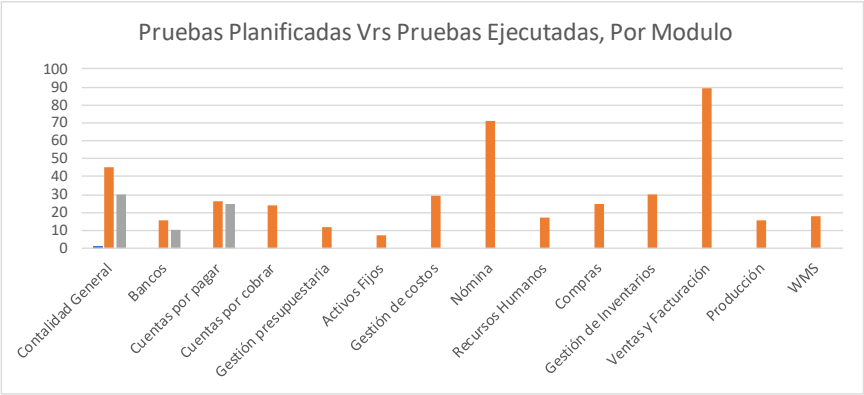
| Módulo                 | Responsable | Escenarios de prueba planificados | Escenarios ejecutados |
|------------------------|-------------|-----------------------------------|-----------------------|
| Contalidad General     |             | 45                                |                       |
| Bancos                 |             | 16                                |                       |
| Cuentas por pagar      |             | 26                                |                       |
| Cuentas por cobrar     |             | 24                                |                       |
| Gestión presupuestaria |             | 12                                |                       |
| Activos Fijos          |             | 7                                 |                       |
| Gestión de costos      |             | 29                                |                       |
| Nómina                 |             | 71                                |                       |
| Recursos Humanos       |             | 17                                |                       |
| Compras                |             | 25                                |                       |
| Gestión de Inventarios |             | 30                                |                       |
| Ventas y Facturación   |             | 89                                |                       |
| Producción             |             | 16                                |                       |
| WMS                    |             | 18                                |                       |



Resumen General de avance de pruebas unitarias

Actualizado al:

| Módulo                 | Escenarios de prueba planificados | Escenarios ejecutados | %   |
|------------------------|-----------------------------------|-----------------------|-----|
| Contabilidad General   | 45                                | 30                    | 67% |
| Bancos                 | 16                                | 10                    | 63% |
| Cuentas por pagar      | 26                                | 25                    | 96% |
| Cuentas por cobrar     | 24                                |                       | 0%  |
| Gestión presupuestaria | 12                                |                       | 0%  |
| Activos Fijos          | 7                                 |                       | 0%  |
| Gestión de costos      | 29                                |                       | 0%  |
| Nómina                 | 71                                |                       | 0%  |
| Recursos Humanos       | 17                                |                       | 0%  |
| Compras                | 25                                |                       | 0%  |
| Gestión de Inventarios | 30                                |                       | 0%  |
| Ventas y Facturación   | 89                                |                       | 0%  |
| Producción             | 16                                |                       | 0%  |
| WMS                    | 18                                |                       | 0%  |
| Total                  | 425                               | 65                    | 15% |



# Ejercicio

## **Caso: Sistema de Reservas de Consultas Médicas en Línea**

Completa tus estadísticas con el plan de pruebas elaborado anteriormente

| Test Suit                  | ID | Tipo    | Test Case                 | PASOS a ejecutar                                                                                                                    | Resultado Esperado                                                                                                                       | Estado |
|----------------------------|----|---------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                            |    |         |                           |                                                                                                                                     |                                                                                                                                          |        |
| 1. Instalador del servicio | 1  | Técnica | Validar objeto Instalador | Verifique el nombre del instalador cumple con el siguiente formato:                                                                 | Nombre del archivo cumple con el formato establecido                                                                                     |        |
|                            |    |         |                           | Haga clic derecho y seleccione la opción de Propiedades                                                                             | La pestaña General de propiedades es mostrada                                                                                            |        |
|                            | 2  | Técnica | Ejecutar instalación      | Haga clic en la pestaña de Detalles y verifique que Product name y Product version son los correctos para esta versión del software | Nombre del producto y versión son las esperadas                                                                                          |        |
|                            |    |         |                           | Haga clic derecho sobre el instalador y seleccione la opción Ejecutar como administrador                                            | El programa instalador inicia a ejecutarse y carga una ventana...                                                                        |        |
|                            |    |         |                           | En la ventana mostrada haga clic en el botón Continuar                                                                              | Ventana de selección de ruta de instalación es mostrada                                                                                  |        |
|                            |    |         |                           | Seleccione la ruta de instalación y haga clic en el botón Aceptar                                                                   | La instalación inicia y en la ventana se muestra una barra de progreso de la instalación, cuando se haya completado mostrará un mensaje. |        |
|                            |    |         |                           | Haga clic en el botón Finalizar y reinicie el sistema.                                                                              | Luego de reiniciado el sistema el servicio se mostrará en el listado de Servicios de Windows.                                            |        |

| Test Suit                     | ID |         | Test Case                      | PASOS a ejecutar                                                                                                                      | Resultado Esperado                                       | Estado |
|-------------------------------|----|---------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|--------|
| 2. Configuración del servicio | 1  | Técnica | Configurar servicio en Windows | Buscar el archivo de configuración Config.ini que se encuentra en el directorio de instalación "C:/.../bin"                           | Archivo Config.ini es encontrado en la ruta esperada     |        |
|                               |    |         |                                | Abrir el archivo con un editor de texto como Notepad++ y realizar modificaciones a las propiedades de configuración y guardar cambios | Archivo Config.ini actualizado y guardado                |        |
|                               |    |         |                                | Ir a los Servicios de Windows y reiniciar el servicio "Nombre del servicio"                                                           | El servicio se reinicia con la configuración actualizada |        |

| Test Suit                  | ID |         | Test Case                                          | PASOS a ejecutar                                                                                                                      | Resultado Esperado | Estado |
|----------------------------|----|---------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------|--------|
| 3. Credenciales POA Oracle | 1  | Técnica | Configurar servicio con credenciales POA correctas | Abrir el archivo de configuración y ubicar la línea ""                                                                                |                    |        |
|                            |    |         |                                                    | Agregar los siguientes valores y guardar                                                                                              |                    |        |
|                            |    |         |                                                    | Ubicarse en los Servicios de windows e Iniciar o reiniciar el Servicio ""                                                             |                    |        |
|                            | 2  | Técnica | Configurar servicio con credenciales no existentes | Ubicarse en el directorio de los archivos de log, abrir el documento "" y verificar que que aparece una linea de conexión exitosa: "" |                    |        |

# Antes de Concluir la Sesión

Comparte tu plan de pruebas



# Tarea para el Viernes Próximo

Una cadena de supermercados ha lanzado una aplicación móvil para el **pago en cajas rápidas**. El sistema permite que los clientes escaneen los productos con la cámara del teléfono, agreguen los artículos a un carrito virtual y luego realicen el pago directamente desde la aplicación. El pago puede hacerse con tarjeta de crédito, débito o billetera electrónica. Una vez confirmado, el sistema genera un código QR que el cliente debe mostrar en la salida para validar la compra.

El sistema tiene reglas específicas:

Cada producto debe mostrar precio, descripción y cantidad disponible en inventario.

No se puede pagar si el carrito está vacío.

El sistema debe rechazar pagos con tarjetas vencidas o con saldo insuficiente.

El código QR generado debe ser único y válido solo por 15 minutos.

Si el cliente intenta salir sin mostrar el código, el sistema debe marcar la compra como no validada.

Además, los administradores del supermercado tienen un módulo para **actualizar inventario**. Este módulo debe impedir que se registre un producto con precio negativo o con cantidad en inventario menor a cero.

El software antes descrito, ya fue desarrollado y necesitamos elaborar un plan con casos de pruebas unitarias.

**Comparte tu plan de pruebas y los indicadores que pueden medirlo.**

# Curso: Ingeniería de Software

## Ingeniería en Sistemas y Ciencias de la Computación

### Clase 10: Evolución del Software

Catedrático: Claudia Alegre

21 de marzo de 2026

# Dinámica de Introducción

Hagamos líneas de tiempo:

1. Teléfono: 1980-1990, 1990-2000, 2000-2010, 2010-2020, 2020-2026
2. Automóvil: 1980-1990, 1990-2000, 2000-2010, 2010-2020, 2020-2026
3. Televisión: 1980-1990, 1990-2000, 2000-2010, 2010-2020, 2020-2026
4. Medios de pago: 1980-1990, 1990-2000, 2000-2010, 2010-2020, 2020-2026
5. Comunicación escrita: 1980-1990, 1990-2000, 2000-2010, 2010-2020, 2020-2026

# Tema

## **Evolución del software**

Procesos de evolución

Evolución dinámica del programa

Mantenimiento del software

Predicción de mantenimiento

Reingeniería de software

Mantenimiento preventivo mediante refactorización

Administración de sistemas heredados

# Indicadores de Logro

- Explica la importancia de la evolución del software para adaptarse a los cambios en el entorno, las tecnologías y las necesidades del usuario.
- Identifica y describe las leyes de Lehman sobre la evolución del software y cómo se aplican a proyectos reales.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Evolución del software

[ARPANET: El Internet Antes del Internet](#)

[El ciclo de vida del software y la evolucion tecnológica](#)

# Teoría: Evolución del software

## **¿Por qué evoluciona el software?**

Adaptarse al entorno operativo

Adaptarse al usuario

Mejorar en el desempeño

Mejorar en funcionalidades

Corrección de errores



# Teoría: Evolución del software

## **Importancia de la evolución**

Los grandes sistemas de software, como los militares o los de infraestructuras, suelen tener una vida útil de 30 años o más. Los sistemas empresariales pueden tener más de 10 años. La longevidad se debe a la necesidad de recuperar la inversión realizada en el desarrollo del software.

Las empresas invierten en cambios de software para mantener el valor de sus activos empresariales críticos. El coste de mantenimiento de los sistemas existentes suele ser mayor que el de desarrollo de los nuevos. Los estudios indican que entre el 60% y el 90% de los costes de software están asociados a la evolución.

# Teoría: Evolución del software

## **Proceso en espiral**

La ingeniería de software sigue un proceso en espiral durante toda la vida útil del sistema. Se crean versiones periódicas para incorporar cambios y actualizaciones basados en la evolución de los requisitos. El desarrollo de versiones posteriores puede comenzar incluso antes de que se publique la versión actual.

En general, los sistemas de software tienen una larga vida útil y requieren una evolución y adaptación continuas para satisfacer las necesidades cambiantes de la empresa y los factores del entorno. La ingeniería del software sigue un proceso en espiral continuo para apoyar estos sistemas en evolución.

# Teoría: Evolución del software

## **Proceso de evolución y mantenimiento (o servicio)**

Una visión alternativa del ciclo de vida de la evolución del software para sistemas empresariales.

Evolución el sistema vive en un ciclo donde se encuentra en su uso operacional y evoluciona cuando los nuevos requisitos son propuestos e implementados en el sistema.

Servicio el software sigue siendo útil pero los únicos cambios realizados son para mantenerlo operativo. Por ejemplo: corrección de errores y cambios producto de modificaciones en el entorno del software. No se agregan nuevas funcionalidades.

Retiro gradual el software puede ser utilizado todavía pero no se realizan más cambios sobre el mismo.

# Teoría: Evolución del software

## **Transición fluida en la evolución del software**

Cuando la misma empresa es responsable del software durante toda su vida útil, se produce una transición fluida del desarrollo a la evolución. Se aplican sistemáticamente los mismos métodos y procesos de desarrollo de software. Este modelo se utiliza habitualmente para productos de software y aplicaciones.

# Teoría: Evolución del software

## **Procesos de evolución**

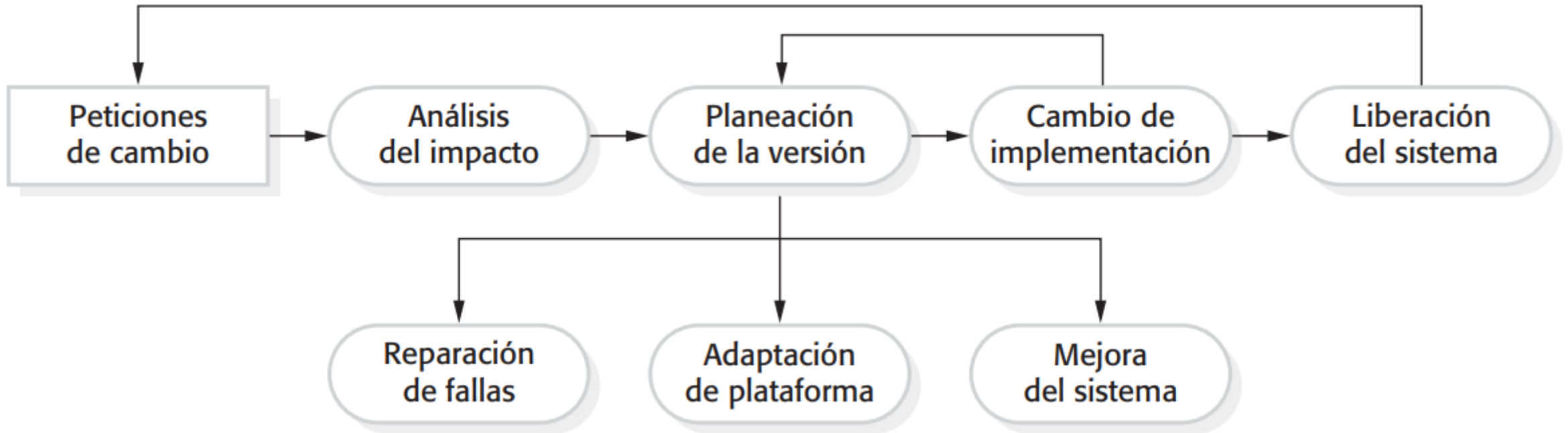
Los procesos de evolución del software varían dependiendo del tipo de software que se mantiene, de los procesos de desarrollo usados en la organización y de las habilidades de las personas que intervienen.

Las propuestas de cambio al sistema son el motor para la evolución del sistema en todas las organizaciones. Estos cambios provienen de:

- requerimientos existentes que no se hayan implementado en el sistema
- nuevos requerimientos
- reportes de bugs
- de nuevas ideas del equipo de desarrollo

# Teoría: Evolución del software

Panorama del proceso de evolución, el cual incluye actividades fundamentales de análisis del cambio, planeación de la versión, implementación del sistema y su liberación a los clientes.



# Teoría: Evolución del software

## **Cambios urgentes**

En ocasiones, las peticiones de cambio se relacionan con problemas del sistema que tienen que enfrentarse de manera urgente. Surgen básicamente por tres razones:

Si ocurre una falla seria del sistema que deba repararse para permitir que continúe la operación normal.

Si los cambios a los sistemas que operan el entorno tienen efectos inesperados que perturban la operación normal.

Si hay cambios no anticipados a la empresa que opera el sistema, como el surgimiento de competidores nuevos o la introducción de una nueva legislación que afecte al sistema.

# Teoría: Evolución del software

## Cambios urgentes

En ocasiones, las peticiones de cambio se relacionan con problemas del sistema que tienen que enfrentarse de manera urgente. Surgen básicamente por tres razones:

- Si ocurre una falla seria del sistema que deba repararse para permitir que continúe la operación normal.
- Si los cambios a los sistemas que operan el entorno tienen efectos inesperados que perturban la operación normal.
- Si hay cambios no anticipados a la empresa que opera el sistema, como el surgimiento de competidores nuevos o la introducción de una nueva legislación que afecte al sistema.

En vez de modificar los requerimientos y el diseño, se puede hacer una **reparación** de emergencia al programa para resolver el problema de inmediato:





# Teoría: Evolución del software

## **Sistemas heredados**

En una organización, los sistemas se remplazan a medida que el negocio cambia.

Sin embargo, muchos viejos sistemas continúan siendo utilizados, e incluso, tienen un rol crítico en el negocio. Estos son llamados sistemas heredados.

No son solo sistemas de software. Abarcan hardware, software, librerías, software de soporte y procesos de negocio.

Han tenido mantenimiento por un largo tiempo, por lo que su estructura puede estar degradada.

Pueden depender de hardware antiguo.

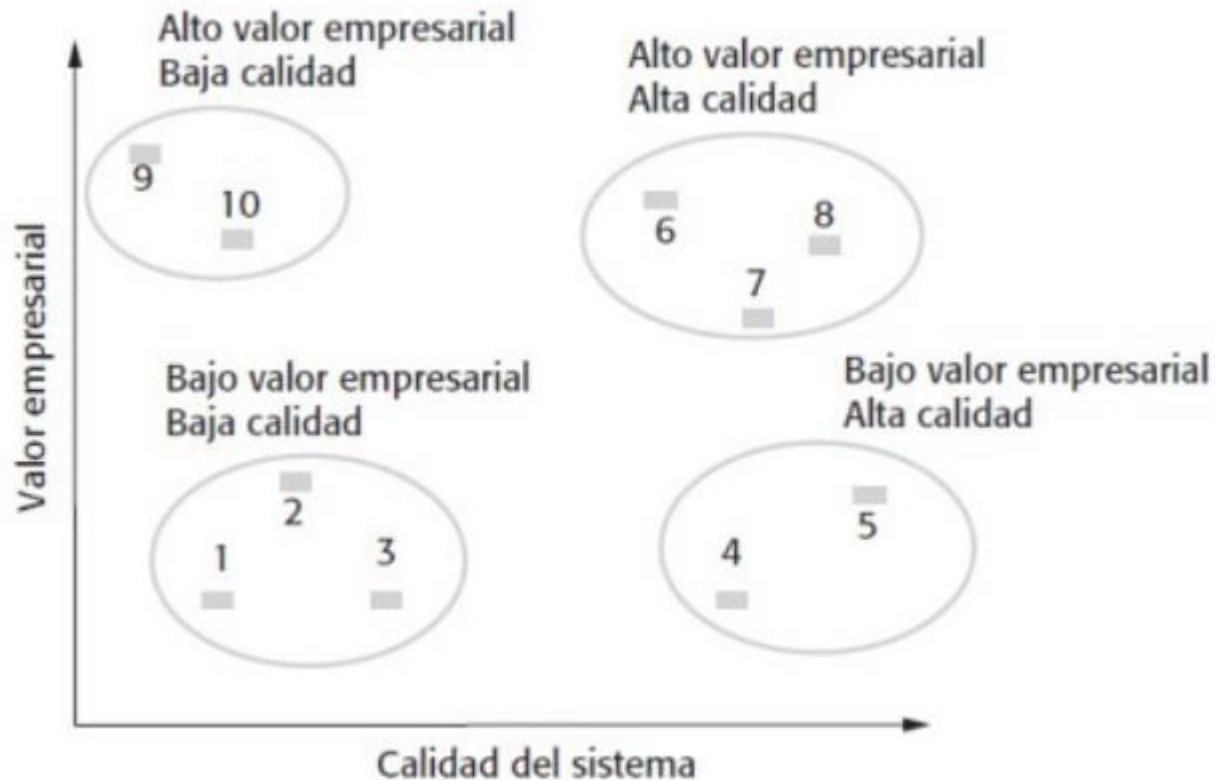
Es probable que no soporten nuevos procesos de negocio.

# Teoría: Evolución del software

El mantenimiento de estos sistemas tiene dificultades y es costoso.

- Falta de habilidades o conocimiento de viejas tecnologías (recursos externos)
- Dificultades para entender el código debido a que fue modificado por muchas personas con diferentes estilos
- Sistema degradado por muchos años de mantenimiento
- Falta documentación o está desactualizada.
- Vulnerabilidades de seguridad
- Problemas para integrarse con sistemas contruidos con tecnologías nuevas
- Ausencia de soporte oficial
- Hardware obsoleto y costoso de mantener
- Problemas a nivel de datos: duplicación y baja calidad.

# Teoría: Categorías de sistemas heredados



- 1,2,3 Estos sistemas deben ser desechados.
- 4,5 Reemplazar con componentes de terceros, desechar completamente o mantener.
- 6,7,8 Continuar con el sistema en operación y realizar un mantenimiento normal del sistema.
- 9,10 Estos hacen una importante contribución al negocio pero son caros de mantener. Debe ser reestructurado o reemplazado por otro sistema comercial que esté disponible.

# Teoría: Mantenimiento de software

Es la modificación de un programa luego de que está siendo utilizado.

El término se utiliza, sobre todo, para realizar cambios en el software personalizado.

Normalmente el mantenimiento no genera grandes cambios en la arquitectura del sistema.

Los cambios implican modificar componentes existentes y agregar nuevos componentes al sistema.

# Teoría: Mantenimiento de software

## **Tipos de mantenimiento**

Mantenimiento para reparar defectos o vulnerabilidades del software

Los errores pueden haber sido introducidos en diferentes etapas, lo que influye en el costo de la corrección.

Mantenimiento para adaptar el software a un entorno operativo diferente

Cambios en el sistema para que este opere en entornos diferentes a los de la implementación inicial.

Mantenimiento para agregar o modificar funcionalidades al sistema

Modificar el sistema para satisfacer nuevos requisitos

# Ejercicio

## **Sistema de Facturación de una PyME**

Una pequeña empresa de alimentos implementó hace 8 años un sistema de facturación en **Visual Basic** con base de datos Access.

Funcionalidad: emisión de facturas, control de clientes y reportes básicos.

Usuarios: personal administrativo y contable.

Problema: el sistema ya no responde a las necesidades actuales de la empresa.

## **Situaciones de mantenimiento**

### **Correctivo**

El sistema genera facturas con errores en el cálculo del IVA.

Pregunta: ¿qué pasos seguirían para identificar y corregir el bug?

### **Adaptativo**

La empresa ahora debe cumplir con nuevas regulaciones fiscales electrónicas.

Pregunta: ¿cómo adaptarían el sistema para emitir facturas electrónicas válidas?

### **Perfectivo**

Los usuarios piden reportes más detallados y exportables a Excel.

Pregunta: ¿qué mejoras implementarían para aumentar la utilidad del sistema?

### **Preventivo**

El sistema está en una tecnología obsoleta (VB + Access).

Pregunta: ¿qué acciones tomarían para prevenir riesgos futuros de compatibilidad y seguridad?

# Teoría: Reingeniería de software

Es la reestructuración o la reescritura de parte o todo un sistema heredado sin cambiar su funcionalidad.

Es aplicable cuando los sistemas necesitan un mantenimiento frecuente.

La reingeniería involucra agregar esfuerzo para hacer que sea más fácil mantener el sistema.

El sistema puede ser reestructurado y redocumentado.

# Teoría: Reingeniería de software

## **Ventajas de la reingeniería**

### Reducción del riesgo

Hay un alto riesgo en el desarrollo de un nuevo software. Pueden haber problemas de desarrollo, problemas de equipo y problemas de especificación.

### Reducción del costo

Generalmente los costos de la reingeniería son mucho menos significantes que los costos de desarrollar nuevo software.



# Teoría: Refactorización

La refactorización es el proceso de hacer mejoras a un programa para reducir la degradación del sistema al realizar cambios.

Se puede pensar en la refactorización como un "mantenimiento preventivo" que reduce los problemas en cambios futuros.

La refactorización trata de modificar un programa para mejorar su estructura, reducir su complejidad o hacerlo más fácil de entender.

Cuando se refactoriza un programa no se deben agregar funcionalidades.

Es inherente a las metodologías ágiles.

.

# Teoría: Refactorización y Reingeniería

La reingeniería toma lugar luego de que el sistema ha sido mantenido por algún tiempo y los costos se incrementan.

Utiliza las herramientas automatizadas para procesar y rediseñar sistemas heredados para crear un nuevo sistema más mantenible.

La refactorización es un proceso continuo de mejora en todo el proceso de desarrollo y en la evolución.

Se tiene la intención de evitar que se incremente la degradación de la estructura y el código para que así no se dificulte el mantenimiento del sistema

# Ejercicio

## **Sistema Bancario de Atención al Cliente**

Un banco regional desarrolló hace 12 años un sistema de atención al cliente en **.NET antiguo** con base de datos SQL Server.

Funcionalidad: registro de clientes, apertura de cuentas, seguimiento de solicitudes.

Problemas actuales:

- Interfaz obsoleta, difícil de usar en dispositivos móviles.

- Integración limitada con otros sistemas (tarjetas, préstamos, banca en línea).

- Código monolítico, difícil de escalar.

- Los reportes tardan horas en generarse.

## **Ejercicio de reingeniería**

### **Diagnóstico inicial**

- Identificar qué partes del sistema siguen siendo útiles (ejemplo: base de datos de clientes).

- Detectar módulos críticos que requieren rediseño completo.

### **Propuesta de reingeniería**

### **Impacto esperado**

# Ejercicio - Resultado

## Ejercicio de reingeniería

### Diagnóstico inicial

- Identificar qué partes del sistema siguen siendo útiles (ejemplo: base de datos de clientes).
- Detectar módulos críticos que requieren rediseño completo.

### Propuesta de reingeniería

- Migrar a una arquitectura moderna basada en **microservicios**.
- Rediseñar la interfaz como **aplicación web responsiva** y app móvil.
- Integrar APIs para conectarse con sistemas de tarjetas y préstamos.
- Implementar reportes en tiempo real con herramientas de análisis de datos.

### Impacto esperado

- Mayor satisfacción del cliente por accesibilidad móvil.
- Reducción de tiempos de respuesta.
- Escalabilidad para soportar más usuarios.
- Cumplimiento con regulaciones modernas de seguridad.

# Antes de Concluir la Sesión

Comparte tus ejercicios de clase

- Línea de tiempo
- Respuestas de mantenimiento
- Propuesta de reingeniería

# Tarea para el Viernes Próximo

## **ERP de una empresa de alimentos**

La empresa “Alimex” implementó hace 15 años un **ERP (Enterprise Resource Planning)** para gestionar inventarios, facturación y recursos humanos.

Tecnología: aplicación monolítica en Java con base de datos Oracle.

Usuarios: administrativos, contadores y gerentes.

Problemas actuales:

- Interfaz obsoleta, poco amigable.

- Lentitud en reportes de inventario.

- Dificultad para integrarse con aplicaciones móviles y sistemas externos.

- Código con deuda técnica acumulada.

Presenta propuesta y construye una **línea de tiempo de evolución del ERP**:

Inicio - Mantenimiento - Refactorización - Reingeniería.



# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación

## Clase 11: Sistemas Sociotécnicos

Catedrático: Claudia Alegre

28 de marzo de 2026

# Dinámica de Introducción

## Paso 1: División en grupos

Forma 2 grupos:

**Grupo A (Tecnología):** representan las máquinas, software y procesos técnicos.

**Grupo B (Personas):** representan los usuarios, roles y comunicación.

## Paso 2: Escenario ficticio

Plantea un caso breve: *“Una universidad instala un sistema digital para registrar asistencia en clase. El software funciona, pero los estudiantes no saben cómo usarlo y los docentes no reciben capacitación.”*

## Paso 3: Debate rápido

Grupo A responde: ¿qué problemas técnicos surgen si el sistema se usa mal?

Grupo B responde: ¿qué problemas sociales aparecen si no hay capacitación ni comunicación?

ANOTEN EN EL PIZARRON LOS RESULTADOS DEL GRUPOA y del GRUPOB



# Dinámica de Introducción

- **Paso 4: Conclusión conjunta**
- Entre todos, identifican que **el sistema falla** si no se integran ambos aspectos.
- Se introduce el concepto: *“Esto es un sistema sociotécnico: la unión inseparable de lo social y lo técnico.”*

# Tema

**Sistemas sociotécnicos**

# Indicadores de Logro

- Explica el concepto de sistemas sociotécnicos y su relación entre componentes técnicos (software, hardware) y sociales (usuarios, procesos, organizaciones).

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Sistemas sociotécnicos

Los sistemas sociotécnicos integran hardware, software y elementos humanos para funcionar eficazmente. Ejemplos como estaciones meteorológicas automatizadas ilustran su complejidad. Estos sistemas se estructuran en siete capas, desde lo físico a lo social, y presentan propiedades emergentes. La variabilidad humana introduce no determinismo, lo que plantea desafíos en diseño e ingeniería, requiriendo adaptabilidad y flexibilidad para el éxito a largo plazo.



# Teoría: Sistemas sociotécnicos

Los sistemas sociotécnicos pueden analizarse a través de una estructura estratificada de siete capas, que van de lo concreto a lo abstracto. La primera capa es la del equipo físico, que incluye todos los dispositivos de hardware. La segunda es la del sistema operativo, que posibilita la interacción entre el hardware y las aplicaciones de software. La tercera capa abarca las comunicaciones y la gestión de datos, proporcionando las interfaces necesarias para la conectividad y el manejo de la información. La cuarta capa es la de las aplicaciones, que suministra las funcionalidades específicas demandadas por los usuarios. La quinta capa corresponde al proceso empresarial, que define los procedimientos y flujos de trabajo organizacionales. La sexta es la de la estructura organizativa, que comprende la cultura, estrategias y políticas de la entidad. La séptima y última capa es la social, que engloba las leyes, normativas y aspectos culturales externos.



# Teoría: Sistemas sociotécnicos

Los sistemas que incorporan software se dividen en dos categorías principales: sistemas técnicos basados en computadora y sistemas sociotécnicos. Los sistemas técnicos se componen exclusivamente de hardware y software, y ejemplos de estos incluyen dispositivos como teléfonos móviles y televisores. En contraste, los sistemas sociotécnicos son más intrincados, ya que integran los componentes técnicos con los usuarios que los operan, los procesos operativos y las restricciones organizacionales y legales. Estos sistemas están sujetos a políticas internas y pueden ser influenciados por leyes y regulaciones externas, lo que añade capas de complejidad en su gestión y operación.



# Teoría: Sistemas sociotécnicos

Los sistemas sociotécnicos exhiben propiedades emergentes, que son atributos del sistema en su totalidad y no pueden ser deducidos de sus componentes individuales. Estas propiedades pueden ser funcionales, como la capacidad de transporte de una bicicleta completamente ensamblada, o no funcionales, como la confiabilidad y la seguridad. Las propiedades no funcionales son de especial importancia en los sistemas basados en computadora, ya que un déficit en estas características puede comprometer la utilidad del sistema en su totalidad.





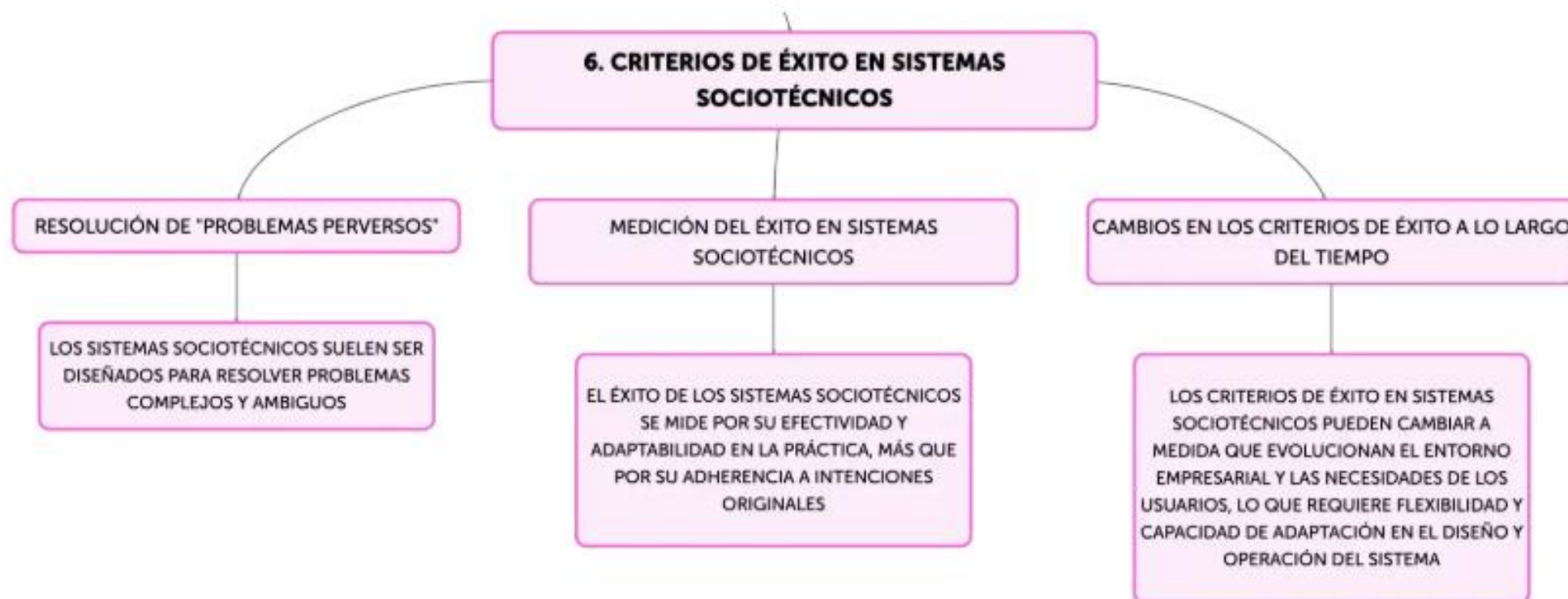
# Teoría: Sistemas sociotécnicos

A diferencia de los sistemas puramente técnicos, que son mayormente deterministas, los sistemas sociotécnicos incluyen el factor humano, que introduce variabilidad y no determinismo. Las reacciones humanas pueden diferir ante las mismas situaciones debido a una multitud de factores emocionales, físicos y contextuales, lo que añade un elemento de imprevisibilidad. Este no determinismo puede ser fuente de errores y fallos, ya que los operadores humanos pueden cometer equivocaciones en su interacción con el sistema, lo que requiere un diseño cuidadoso y consideración de la interfaz humano-máquina.



# Teoría: Sistemas sociotécnicos

Los sistemas sociotécnicos suelen ser diseñados para resolver "problemas perversos", que son complejos y carecen de una definición clara y definitiva. El éxito de estos sistemas no se mide tanto por la adherencia a las intenciones originales, sino por su efectividad y adaptabilidad en la práctica. Los criterios de éxito pueden cambiar a lo largo del tiempo, ya que el entorno empresarial y las necesidades de los usuarios evolucionan, lo que exige flexibilidad y capacidad de adaptación en el diseño y operación del sistema.



# Teoría: Sistemas sociotécnicos

La ingeniería de sistemas sociotécnicos implica una serie de actividades que abarcan desde la adquisición hasta el mantenimiento del sistema. Los ingenieros deben considerar todos los aspectos, incluyendo el hardware, el software, la interacción con los usuarios, el contexto operativo y los servicios proporcionados por el sistema. La vida útil de un sistema sociotécnico se divide en fases de adquisición, desarrollo y operación, cada una con sus propios retos y metodologías. La fase operativa es particularmente crítica, ya que los procesos deben reflejar y adaptarse al entorno de trabajo real, así como a los cambios y nuevos requerimientos que surjan con el tiempo



# Ejercicio

## Caso: “Plataforma de Tutorías entre Estudiantes”

Un grupo de jóvenes de secundaria y universidad quiere crear una aplicación que les permita ofrecer y recibir tutorías en distintas materias. La idea es que los estudiantes que dominan un tema puedan registrarse como tutores, mientras que otros se inscriben como aprendices. La plataforma permite agendar sesiones, llevar un registro de horas de estudio y compartir materiales digitales.

El sistema no depende únicamente de la tecnología (la aplicación, el calendario, los archivos compartidos), sino también de los aspectos sociales: la confianza entre estudiantes, la motivación para enseñar y aprender, y las reglas de convivencia que aseguran un ambiente respetuoso. Por eso se concibe como un **sistema sociotécnico**, donde lo social y lo técnico se desarrollan en conjunto.

### Ciclo de desarrollo sociotécnico

- **Análisis del contexto social y técnico**
- **Diseño conjunto**
- **Implementación tecnológica**
- **Prueba piloto social**
- **Evaluación y retroalimentación**
- **Ajustes y mejora continua**

Desarrolla el ciclo de desarrollo sociotécnico en cada etapa

# Ejercicio Resultado

## **Ciclo del desarrollo sociotécnico**

### **Análisis del contexto social y técnico**

Se identifican las necesidades: apoyo académico, flexibilidad de horarios, reconocimiento de horas de tutoría.

Se estudian las capacidades técnicas: aplicación móvil, videollamadas, almacenamiento de materiales.

### **Diseño conjunto**

Entidades: Usuarios, Sesiones, Materiales, Evaluaciones.

Reglas sociales: respeto en la comunicación, validación de horas de tutoría, incentivos (puntos o certificados).

### **Implementación tecnológica**

Se desarrolla la app con registro, agenda de sesiones y repositorio de materiales.

Se integran funciones de seguridad y autenticación.

### **Prueba piloto social**

Se invita a un grupo de estudiantes a usar la plataforma.

Se observan comportamientos: participación, calidad de tutorías, problemas de coordinación.

### **Evaluación y retroalimentación**

Métricas técnicas: estabilidad de videollamadas, carga de archivos.

Opiniones sociales: satisfacción, confianza, utilidad percibida.

### **Ajustes y mejora continua**

Corrección de fallos técnicos.

Adaptación de reglas sociales (por ejemplo, sistema de reputación para tutores).

# Antes de Concluir la Sesión

Comparte tus ejercicios de clase

# Tarea para el Viernes 10 de abril

## **Caso: “Plataforma de Bienestar Digital para Estudiantes”**

Un grupo de jóvenes universitarios desarrolla una aplicación móvil que combina **rutinas de estudio, salud mental y actividades sociales**. La idea es que los estudiantes puedan organizar su tiempo de clases, recibir recordatorios de descanso, participar en grupos de apoyo y acceder a recursos de bienestar como meditaciones guiadas o consejos de nutrición.

La aplicación no funciona solo como tecnología: depende de la **participación social** de los estudiantes, la confianza en compartir experiencias personales y la coordinación con tutores y psicólogos voluntarios. Por eso se concibe como un **sistema sociotécnico**, donde lo técnico (app, algoritmos de recomendación, seguridad de datos) y lo social (motivación, cultura de apoyo, reglas de convivencia) se diseñan en conjunto.

**Desarrolla el ciclo de desarrollo sociotécnico en cada etapa**



# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación

## Clase 13: Ingeniería de sistemas

Catedrático: Claudia Alegre

11 de abril de 2026



# Dinámica de Introducción

¿Qué tienen en común un videojuego, un semáforo y una red social?

Nacen, funcionan, fallan y cambian

## **Hagamos grupos:**

Grupo 1: Elijan un video juego

Grupo 2: Un semáforo inteligente

Grupo 3: Whatsapp

Grupo 4: Elijan un sistema operativo

## **Cada grupo debe responder:**

¿Cómo creen que nació ese sistema?

¿Cómo se construyó o diseñó?

¿Cómo funciona en la vida diaria?

¿Qué errores humanos pueden ocurrir?

¿Cómo ha cambiado con el tiempo?

¿Cómo creen que nació ese sistema? (procuración).

¿Cómo se construyó o diseñó? (desarrollo).

¿Cómo funciona en la vida diaria? (operación).

¿Qué errores humanos pueden ocurrir? (error humano).

¿Cómo ha cambiado con el tiempo? (evolución).

# Tema

## **Ingeniería de sistemas**

- Procuración del sistema
- Desarrollo del sistema
- Operación del sistema
- Error humano
- Evolución del sistema

# Indicadores de Logro

- Explica los fundamentos de la ingeniería de sistemas, incluyendo su enfoque interdisciplinario para el diseño, desarrollo y gestión de sistemas complejos.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: Ingeniería de sistemas

| Etapa              | Descripción                                                                                   |
|--------------------|-----------------------------------------------------------------------------------------------|
| <b>Procuración</b> | Identificación de necesidades, definición de requisitos, planeación de recursos y viabilidad. |
| <b>Desarrollo</b>  | Diseño, construcción, pruebas y validación del sistema.                                       |
| <b>Operación</b>   | Implementación, mantenimiento y mejora continua del sistema.                                  |

# Procuración

## **Elementos Clave de la Procuración**

**Análisis de necesidades:** ¿Qué problema debe resolver el sistema?

**Definición de requisitos:** Funcionales (qué debe hacer) y no funcionales (rendimiento, seguridad, escalabilidad).

**Evaluación de viabilidad:** Técnica, económica y organizacional.

**Planificación de recursos:** Hardware, software, personal especializado, presupuesto.

**Gestión de riesgos iniciales:** Identificación de posibles obstáculos y estrategias de mitigación.

# Procuración

## **Riesgos si se omite o se hace mal**

- Requisitos incompletos : sistemas que no satisfacen al usuario.
- Subestimación de recursos: retrasos y sobrecostos.
- Falta de análisis de viabilidad: proyectos que fracasan antes de operar.

# Procuración

EJEMPLO: La procuración de un sistema para una empresa de alimentos podría incluir:

- **Requisitos:** Control de inventarios, trazabilidad de productos, cumplimiento con normas sanitarias.
- **Recursos:** Servidores locales o en la nube, software ERP, capacitación del personal.
- **Viabilidad:** Evaluar costos frente a beneficios en eficiencia logística y satisfacción del cliente.



# Procuración - EJERCICIO

## Caso: Sistema de Gestión de Historias Clínicas en un Hospital Universitario

Un hospital universitario en Guatemala desea modernizar su manejo de historias clínicas, actualmente almacenadas en papel y en archivos dispersos. La dirección busca un sistema de información que centralice, digitalice y asegure el acceso a los datos de pacientes.

### Elementos para la Procuración

#### Necesidades identificadas:

- Acceso rápido y seguro a la información de pacientes.
- Integración con laboratorios y farmacia.
- Cumplimiento con normativas de protección de datos.
- Reducción de errores por duplicidad o pérdida de información.

#### Requisitos iniciales:

- Funcionales: registro de pacientes, historial médico, resultados de laboratorio, recetas electrónicas.
- No funcionales: seguridad, disponibilidad 24/7, escalabilidad para más usuarios.

#### Recursos requeridos:

- Servidores locales o en la nube.
- Software especializado en gestión hospitalaria.
- Capacitación del personal médico y administrativo.
- Presupuesto para licencias y mantenimiento.

#### Viabilidad:

- Técnica:** disponibilidad de infraestructura tecnológica.
- Económica:** análisis costo-beneficio frente a reducción de errores y eficiencia.
- Organizacional:** aceptación del personal y adaptación de procesos.

#### Riesgos iniciales:

- Resistencia al cambio por parte del personal.
- Posibles fallos de seguridad en el manejo de datos sensibles.
- Sobrecostos si no se definen bien los requisitos.

# Procuración - EJERCICIO

**Grupos:**

**1- Usuarios finales**

**2- Administrador del hospital**

**3- Equipo Técnico**

**4- Auditores de cumplimiento legal**

Cada grupo presenta sus necesidades

De manera individual hacer una lista de las necesidades funcionales y no funcionales de todos los grupos

# Procuración - EJERCICIO

## Solución – Fase de Procuración del Sistema de Información Hospitalario

### 1. Análisis de Necesidades

Reuniones con médicos, enfermeras, administrativos y pacientes para identificar problemas actuales (pérdida de expedientes, duplicidad de datos, lentitud en acceso).

Documentación de procesos críticos: admisión, consulta, laboratorio, farmacia.

### 2. Definición de Requisitos

#### Funcionales:

Registro único de pacientes.

Consulta y actualización de historia clínica.

Integración con laboratorio y farmacia.

Generación de reportes médicos y administrativos.

#### No funcionales:

Seguridad y confidencialidad de datos.

Disponibilidad 24/7.

Escalabilidad para más usuarios y servicios.

### 3. Evaluación de Viabilidad

**Técnica:** revisión de infraestructura actual y capacidad de red.

**Económica:** análisis costo-beneficio (reducción de errores, ahorro en papel, eficiencia).

**Organizacional:** disposición del personal para capacitarse y adaptarse.

### 4. Planificación de Recursos

**Tecnológicos:** servidores, software hospitalario, red segura.

**Humanos:** equipo de TI, médicos líderes de proyecto, capacitadores.

**Financieros:** presupuesto inicial y plan de mantenimiento anual.

### 5. Gestión de Riesgos Iniciales

**Resistencia al cambio:** talleres de sensibilización.

**Seguridad de datos:** políticas claras de acceso y respaldo.

**Sobrecostos:** control estricto del alcance y cronograma.

# Error humano

Son fallos derivados de la interacción de los usuarios, operadores o administradores con el sistema.

## **Ejemplos comunes:**

- Ingreso incorrecto de datos (errores tipográficos en historias clínicas).
- Configuración inadecuada de parámetros del sistema.
- Omisión de pasos en procesos críticos (no registrar un medicamento).
- Uso indebido por falta de capacitación.

**Impacto:** Puede comprometer la seguridad, la confiabilidad y la calidad de la información.

## **Gestión:**

- Interfaces intuitivas y con validaciones.
- Capacitación continua.
- Automatización de tareas repetitivas.
- Monitoreo y retroalimentación para detectar patrones de error.

# Evolución del sistema

**Definición:** Es la adaptación y mejora continua del sistema a lo largo de su ciclo de vida, respondiendo a cambios en el entorno, en la organización y en los usuarios.

## **Factores que impulsan la evolución:**

- Nuevas necesidades de los usuarios.
- Cambios normativos o legales.
- Avances tecnológicos.
- Lecciones aprendidas de errores humanos y fallos operativos.

## **Formas de evolución:**

- **Correctiva:** ajustes para reparar errores detectados.
- **Adaptativa:** modificaciones para responder a cambios externos (ej. nuevas regulaciones sanitarias).
- **Perfectiva:** mejoras de rendimiento, usabilidad o seguridad.
- **Preventiva:** cambios para reducir riesgos futuros.

## Relación entre Error Humano y Evolución

El error humano no es solo una falla: es una **f fuente de aprendizaje**. Cada error detectado puede convertirse en un **requisito de evolución** del sistema.

Ejemplo: si los médicos olvidan registrar alergias en el sistema, la evolución puede incluir un **campo obligatorio con alerta automática**.

Así, el sistema se vuelve más robusto y resiliente, integrando la experiencia real de los usuarios.

# Hagamos un ejercicio – Seguimos con el hospital universitario

Grupo A: Errores humanos en el uso del sistema (ej. ingreso de datos incompletos).

Grupo B: analiza esos errores y propone mejoras evolutivas (ej. validaciones, alertas, rediseño de interfaz).

De manera individual presentan su matriz de errores y mejoras evolutivas

# Antes de Concluir la Sesión

Comparte tus ejercicios de clase



# Tarea para el Viernes 17 de abril

## Caso: Sistema de Información para una Biblioteca Universitaria

### 1. Procuración del Sistema

La universidad detecta que su biblioteca tiene problemas de gestión:

- Los préstamos se registran en hojas de cálculo dispersas.
- No hay control de devoluciones ni multas.
- Los estudiantes no pueden consultar disponibilidad en línea.

#### Acciones de procuración:

Reuniones con bibliotecarios, estudiantes y administración para levantar necesidades.

Definición de requisitos: catálogo digital, gestión de préstamos, alertas de vencimiento, acceso web.

Estudio de factibilidad técnica y económica.

Plan de recursos: servidores, software de gestión bibliotecaria, capacitación del personal.

### 2. Desarrollo del Sistema

Se diseña e implementa un **sistema web** con:

- Módulo de catálogo digital con buscador.
- Módulo de préstamos y devoluciones con alertas automáticas.
- Integración con cuentas de estudiantes.
- Reportes de uso y estadísticas para la administración.

**Metodología:** desarrollo ágil con pruebas piloto en una facultad antes de escalar a toda la universidad.

### 3. Operación del Sistema

Una vez implementado:

- Los estudiantes consultan libros en línea y reservan ejemplares.
- Los bibliotecarios registran préstamos y devoluciones en el sistema.
- El sistema envía correos de recordatorio antes de la fecha de vencimiento.
- La administración recibe reportes mensuales de uso.

### 4. Error Humano

Durante la operación aparecen errores:

- Bibliotecarios olvidan registrar devoluciones, generando multas injustas.
- Estudiantes ingresan mal sus credenciales y creen que el sistema no funciona.
- Personal administrativo borra registros por accidente.

#### Gestión del error humano:

Validaciones en la interfaz (ej. confirmación antes de borrar registros).

Capacitación periódica.

Manuales de uso y soporte técnico accesible.

### 5. Evolución del Sistema

El sistema se adapta con el tiempo:

- Se añade un módulo de **préstamos digitales de e-books**.
- Se integra con aplicaciones móviles para notificaciones push.
- Se mejora la seguridad con autenticación multifactor.
- Se ajustan procesos según retroalimentación de usuarios.

**Redactar un informe** que describa cada fase (procuración, desarrollo, operación, error humano y evolución).

**Identificar riesgos y beneficios** en cada etapa.

**Proponer mejoras evolutivas** basadas en los errores humanos detectados.



# Curso: Ingeniería de Software

## Ingeniería en Sistemas y Ciencias de la Computación

### Clase 15: Confiabilidad y seguridad

Catedrático: Claudia Alegre

25 de abril de 2026

# Dinámica de Introducción

- Inyección de código
- Ataques DDoS

# Dinámica de Introducción

## Casos de Inyección de Código (SQL Injection)

### Heartland Payment Systems (2008)

**Impacto:** 130 millones de números de tarjetas de crédito y débito expuestos.

**Consecuencia:** Pérdidas de más de **\$140 millones** en multas y demandas.

**Lección:** Validar entradas y monitorear bases de datos es esencial.

### Sony PlayStation Network (2011)

**Impacto:** 77 millones de cuentas comprometidas.

**Consecuencia:** Pérdidas estimadas en **\$170 millones** y suspensión del servicio por semanas.

**Lección:** Incluso grandes corporaciones son vulnerables si no protegen sus aplicaciones web.

### Yahoo! Voices (2012)

**Impacto:** Medio millón de correos y contraseñas filtrados.

**Lección:** La falta de protección en aplicaciones heredadas expone datos masivos.

### TalkTalk (2015)

**Impacto:** Datos de 157,000 clientes comprometidos.

**Consecuencia:** Pérdidas de £60 millones y gran daño reputacional.

# Dinámica de Introducción

Casos de Ataques DDoS

**Azure (2021)**

**Impacto:** Ataque de 3.47 Tbps, el más grande registrado en ese momento.

**Lección:** La nube también es blanco de ataques masivos y requiere defensas distribuidas

# Tema

## **Confiabilidad y seguridad**

Propiedades de confiabilidad

Disponibilidad y fiabilidad

Protección

Seguridad

Reutilización de software

Panorama de la reutilización

Frameworks de aplicación

Líneas de productos de software

Reutilización de productos COTS

# Indicadores de Logro

- El estudiante es capaz de identificar de manera precisa las principales amenazas a la confiabilidad y seguridad en un sistema de software, incluyendo fallos potenciales y vulnerabilidades comunes como inyección de código, ataques DDoS, o problemas de integridad de datos.

# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea





Teoría: Propiedades de confiabilidad

# Teoría: Propiedades de confiabilidad

- Protección

Los sistemas críticos para la protección son aquellos en los que resulta esencial que la operación del sistema sea segura en todo momento; esto es, el sistema nunca debe dañar a las personas o a su entorno, incluso cuando falle. Los ejemplos de sistemas críticos para la protección incluyen los sistemas de control y monitorización en las aeronaves, los sistemas de control de procesos en plantas químicas y farmacéuticas, y los sistemas de control de automotores.

# Teoría: Propiedades de confiabilidad

## Seguridad

La seguridad es un atributo del sistema que refleja la habilidad de éste para protegerse a sí mismo de ataques externos, que podrían ser accidentales o deliberados. Estos ataques externos son posibles puesto que la mayoría de las computadoras de propósito general ahora están en red y, en consecuencia, son accesibles a personas externas.



# Teoría: Propiedades de confiabilidad

- Disponibilidad y fiabilidad

La disponibilidad y fiabilidad del sistema son propiedades estrechamente relacionadas que se expresan también como probabilidades numéricas. La disponibilidad de un sistema es la probabilidad de que el sistema funcionará y ejecutará para entregar a los usuarios sus servicios bajo pedido. La fiabilidad de un sistema es la probabilidad de que los servicios del sistema se entregarán como se definió en la especificación del sistema

# Teoría: Panorama de la reutilización

Seleccionar una técnica adecuada depende de los requerimientos del sistema a desarrollar, la tecnología y los activos reutilizables disponibles, y la experiencia del equipo de desarrollo. Los factores clave que deben considerarse al planear la reutilización son:

- El calendario de desarrollo para el software

Si el software debe desarrollarse rápidamente, usted debe tratar de reutilizar sistemas comerciales en vez de componentes individuales.

- La vida esperada del software

Si usted desarrolla un sistema de prolongada duración, debe enfocarse en la capacidad de mantenimiento del sistema.

- Los antecedentes las habilidades y la experiencia del equipo de desarrollo

Todas las tecnologías de reutilización son bastante complejas, y es necesario mucho tiempo para entenderlas y usarlas de manera efectiva.

- La criticidad del software y sus requerimientos no funcionales

Para un sistema crítico que deba certificarse mediante un regulador externo, quizá deba crear un caso de confiabilidad para el sistema. Esto es difícil si no tiene acceso al código fuente del software.

- El dominio de aplicación

En algunos dominios de aplicación, tales como los sistemas de fabricación e información médica, existen muchos productos genéricos que pueden reutilizarse al configurarlos para una situación local

- La plataforma en la que operará el sistema

Algunos modelos de componentes, como .NET, se especifican para plataformas Microsoft. De igual modo, sistemas genéricos de aplicación pueden ser específicos de plataforma y sólo podrá reutilizarlos si su sistema está diseñado para la misma

# Frameworks

Un framework es una estructura genérica que se extiende para crear una aplicación o un subsistema más específico.

*un conjunto integrado de artefactos de software (tales como clases, objetos y componentes), que colaboran en la facilitación de una arquitectura de reutilización para una familia de aplicaciones relacionadas. (2004 Schmidt y sus colaboradores)*

Los frameworks brindan soporte para características genéricas que es probable que sean utilizadas en todas las aplicaciones de un tipo similar

# Frameworks

- Los frameworks se implementan como una colección de clases de objetos concretos y abstractos en un lenguaje de programación orientado a objetos.
- Los frameworks son específicos del lenguaje
- Existen frameworks disponibles en todos los lenguajes de programación orientados a objetos de uso común Java, C#, C++, así como lenguajes dinámicos como Ruby y Python
- Un framework puede incorporar muchos otros frameworks
- Es posible usar un framework para crear una aplicación completa o implementar parte de una aplicación
- Frameworks de Aplicaciones Web (WAF)

# Líneas de productos de software

Son familias de aplicación, es un conjunto de aplicaciones con una arquitectura común y componentes compartidos, con cada aplicación especializada para reflejar diferentes requerimientos.

Las líneas de producto de software surgen por lo general de aplicaciones existentes. Esto es, una organización desarrolla una aplicación y, luego, cuando se requiere un sistema similar, el código de ésta se reutiliza de manera informal en la nueva aplicación



# COTS

- Un producto COTS (Comercial-Off-The-Shelf ) es un sistema de software que puede adaptarse a las necesidades de diferentes clientes sin cambiar el código fuente del sistema, este software se diseña para uso general, incluye regularmente muchas características y funciones.
- Los productos COTS se adaptan al usar mecanismos de configuración internos que permiten que la funcionalidad del sistema se adecue a necesidades específicas del cliente.

# EJERCICIO

3 Ejemplos de Frameworks

3 Ejemplos de Líneas de productos

3 Ejemplos de productos COTS

# Antes de Concluir la Sesión

Comparte tus ejercicios de clase

# Tarea para el Viernes 1 de mayo

## Caso: Empresa de Comercio Electrónico “Mercado Digital”

La empresa quiere implementar un sistema para:

- Administrar catálogo de productos.
- Procesar pagos en línea.
- Gestionar inventarios y envíos.
- Ofrecer un portal web y aplicación móvil para clientes.

### Opciones disponibles

#### Framework

Usar **Laravel (PHP)** o **Ruby on Rails** para construir la plataforma desde cero.

**Ventaja:** Flexibilidad total, se adapta exactamente al modelo de negocio.

**Desventaja:** Mayor tiempo de desarrollo y necesidad de un equipo técnico especializado.

#### Línea de productos de software

Adoptar **Adobe Commerce (Magento)** o **Salesforce Commerce Cloud**.

**Ventaja:** Conjunto de herramientas integradas para marketing, ventas y gestión de clientes.

**Desventaja:** Puede ser costoso y requiere capacitación para aprovechar todas las funciones.

#### COTS (Commercial Off-The-Shelf)

Usar una solución lista como **Shopify** o **WooCommerce**.

**Ventaja:** Rápida implementación, soporte técnico y actualizaciones automáticas.

**Desventaja:** Limitaciones en personalización y dependencia de la plataforma.

## ACTIVIDADES A REALIZAR:

**Paso 1:** Analizar las tres opciones

**Paso 2:** Elegir cuál recomendarían a la empresa según sus necesidades.

**Paso 3:** Justificar su elección respondiendo:

¿Cuál opción ofrece mayor rapidez de implementación?

¿Cuál opción brinda más flexibilidad para crecer en el futuro?

¿Qué riesgos de seguridad y confiabilidad existen en cada caso?

¿Qué impacto tendría en los costos y en la experiencia del cliente?

# Curso: Ingeniería de Software Ingeniería en Sistemas y Ciencias de la Computación



## Clase 16: Ingeniería de software basada en componentes

Catedrático: Claudia Alegre

2 de mayo de 2026

# Dinámica de Introducción

Diferencia entre  
componente y librería

| Aspecto         | Componente                                                                                                                                 | Librería                                                                                   |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Definición      | Unidad de software <b>independiente y reutilizable</b> que ofrece una funcionalidad específica mediante <b>interfaces bien definidas</b> . | Conjunto de <b>funciones o clases</b> que el desarrollador puede usar dentro de su código. |
| Encapsulamiento | Encapsula tanto la lógica como los datos y define cómo interactuar con otros componentes.                                                  | Encapsula solo código reutilizable; depende del programa que la usa para su contexto.      |
| Comunicación    | Interactúa con otros componentes mediante <b>interfaces o contratos</b> (por ejemplo, servicios o APIs).                                   | Se invoca directamente desde el código fuente mediante llamadas a funciones o métodos.     |
| Independencia   | Puede funcionar como parte de un sistema mayor sin conocer los detalles internos de otros componentes.                                     | Depende del entorno de ejecución y del lenguaje; no suele ser independiente.               |
| Ejemplo         | Un módulo de autenticación que se integra en distintos sistemas mediante una API.                                                          | Una librería de cifrado (como OpenSSL) que provee funciones para encriptar datos.          |
| Reutilización   | Diseñado para ser <b>plug-and-play</b> : se puede integrar fácilmente en diferentes aplicaciones.                                          | Reutilizable a nivel de código, pero requiere integración manual y adaptación.             |

# Tema

## **Ingeniería de software basada en componentes**

Componentes y modelos de componentes

Modelos de componentes

Procesos CBSE

CBSE para reutilización

CBSE con reutilización

# Indicadores de Logro

- El estudiante es capaz de diseñar sistemas modulares utilizando componentes reutilizables, asegurando una arquitectura bien definida que promueva la escalabilidad y el mantenimiento eficiente del software.



# Actividades

- Clase magistral
- Ejercicios en clase
- Tarea

# Teoría: CBSE

## **Ingeniería de Software Basada en Componentes (CBSE)**

Es un enfoque de desarrollo de software que se centra en construir sistemas a partir de **componentes reutilizables**. La idea es similar a la construcción con piezas de Lego: en lugar de programar todo desde cero, se ensamblan módulos ya probados y diseñados para funcionar juntos

# Teoría: CBSE

## **Componentes y modelos de componentes**

**Componente:** Unidad de software independiente que ofrece una funcionalidad específica y puede integrarse en diferentes sistemas. Ejemplo: un módulo de autenticación, un motor de búsqueda, un sistema de pagos.

**Modelos de componentes:** Definen cómo deben diseñarse, describirse y comunicarse los componentes. Incluyen reglas sobre:

- Interfaces (cómo se conectan con otros componentes).
- Contratos (qué servicios ofrecen y qué esperan).
- Ciclo de vida (instalación, ejecución, actualización).

# Teoría: CBSE

## Procesos CBSE

El proceso de desarrollo basado en componentes incluye:

**Análisis de requisitos** identificar qué funcionalidades se necesitan.

**Selección de componentes** buscar módulos existentes que cumplan con los requisitos.

**Adaptación** ajustar los componentes para que encajen en el sistema.

**Integración** ensamblar los componentes en una arquitectura coherente.

**Validación** probar que el sistema funciona correctamente.

# Teoría: CBSE

## **CBSE para reutilización**

El objetivo principal es **maximizar la reutilización**:

- Reducir costos de desarrollo.

- Mejorar la calidad (porque los componentes ya han sido probados).

- Acelerar la entrega de software.

## **CBSE con reutilización**

Aquí se distinguen dos enfoques:

**Desarrollo con reutilización:** se construye un sistema nuevo usando componentes existentes.

**Desarrollo para reutilización:** se crean componentes pensando en que otros proyectos los usarán en el futuro.

```

mirror_mod = modifier_ob.modifiers[0]
#set mirror object to mirror_mod.mirror_object
operation == "MIRROR_X":
    mirror_mod.use_x = True
    mirror_mod.use_y = False
    mirror_mod.use_z = False
operation == "MIRROR_Y":
    mirror_mod.use_x = False
    mirror_mod.use_y = True
    mirror_mod.use_z = False
operation == "MIRROR_Z":
    mirror_mod.use_x = False
    mirror_mod.use_y = False
    mirror_mod.use_z = True

#selection at the end -add
mirror_ob.select= 1
modifier_ob.select=1
context.scene.objects.active = mirror_ob
print("Selected" + str(modifier_ob.name))
mirror_ob.select = 0
#set mirror object to mirror_mod.mirror_object
#set mirror object to mirror_mod.mirror_object
data.objects[one.name].select= 1
print("please select exactly one mirror")

-- OPERATOR CLASSES -----

class MirrorOperator(bpy.types.Operator):
    """Mirror X mirror to the selected object.mirror_mirror_x"""
    bl_idname = "mirror_x"
    bl_label = "Mirror X"

    @classmethod
    def poll(cls, context):
        if context.active_object is not None:

```

CBSE busca que el software se construya como un **ecosistema de piezas intercambiables**, lo que permite mayor eficiencia y calidad.

# Teoría: CBSE

## **Modelos de componentes más comunes**

**CORBA (Common Object Request Broker Architecture):** estándar para comunicación entre objetos distribuidos.

**COM/COM+ (Component Object Model):** modelo de Microsoft para reutilización en Windows.

**EJB (Enterprise JavaBeans):** modelo de componentes para aplicaciones empresariales en Java.

**.NET Components:** modelo de Microsoft basado en el framework .NET.

# Modelos de componentes

## Modelos de componentes

### 1. CORBA (Common Object Request Broker Architecture)

**Origen:** Estándar de la OMG (Object Management Group).

**Función:** Permite que objetos distribuidos escritos en distintos lenguajes se comuniquen.

**Ejemplo:** Un sistema bancario donde módulos en C++ y Java interactúan.

### 2. COM/COM+ (Component Object Model)

**Origen:** Microsoft.

**Función:** Modelo para crear componentes reutilizables en Windows.

**Ejemplo:** Un componente de impresión que puede ser usado por distintas aplicaciones.

### 3. EJB (Enterprise JavaBeans)

**Origen:** Java EE.

**Función:** Modelo para aplicaciones empresariales distribuidas.

**Ejemplo:** Un componente de gestión de usuarios en un sistema de recursos humanos.

### 4. .NET Components

**Origen:** Microsoft .NET Framework.

**Función:** Componentes que se integran en aplicaciones Windows y web.

**Ejemplo:** Un componente de acceso a bases de datos SQL Server.



# Comparación de Modelos de componentes

| Modelo                                                      | Origen / Plataforma              | Función principal                                              | Ejemplo de uso                                | Ventajas                                                 | Desventajas                                               |
|-------------------------------------------------------------|----------------------------------|----------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------|-----------------------------------------------------------|
| <b>CORBA</b><br>(Common Object Request Broker Architecture) | OMG<br>(Object Management Group) | Comunicación entre objetos distribuidos en distintos lenguajes | Sistema bancario con módulos en C++ y Java    | Independencia de lenguaje y plataforma                   | Complejo de implementar, rendimiento limitado             |
| <b>COM/COM+</b><br>(Component Object Model)                 | Microsoft<br>(Windows)           | Creación de componentes reutilizables en Windows               | Componente de impresión usado por varias apps | Integración nativa en Windows, buena compatibilidad      | Dependiente de Windows, limitado fuera de ese entorno     |
| <b>EJB</b><br>(Enterprise JavaBeans)                        | Java EE<br>(Oracle)              | Desarrollo de aplicaciones empresariales distribuidas          | Gestión de usuarios en un sistema de RRHH     | Escalabilidad, soporte para transacciones y seguridad    | Configuración compleja, curva de aprendizaje alta         |
| <b>.NET Components</b>                                      | Microsoft<br>(.NET Framework)    | Componentes para aplicaciones Windows y web                    | Acceso a bases de datos SQL Server            | Amplio ecosistema, soporte multiplataforma con .NET Core | Dependencia del ecosistema Microsoft, tamaño de framework |



# Ejercicio CBSE

## Sistema de Biblioteca Digital

Una universidad quiere implementar un sistema de **biblioteca digital**. El objetivo es que los estudiantes puedan:

- Buscar libros.
- Reservar ejemplares.
- Acceder a material digital (PDFs, artículos).
- Realizar pagos de multas en línea.

**Actividad** - En grupos, deben **diseñar conceptualmente** el sistema usando **CBSE**.

### Pasos de la actividad

#### Identificación de componentes (10 min )

Los estudiantes deben listar qué componentes serían necesarios. Ejemplos:

- Componente de autenticación (login de usuarios).
- Componente de búsqueda (motor de búsqueda de libros).
- Componente de reservas.
- Componente de pagos (PayPal/Stripe).
- Componente de visualización de documentos digitales.

#### Selección de modelo de componentes (5-10 min)

Cada grupo debe utilizar un modelo (CORBA, COM+, EJB, .NET) y justificar por qué sería adecuado para su sistema.

Ejemplo: EJB si quieren un sistema empresarial distribuido en Java.

#### Diseño de integración (10 min)

**Dibujar un esquema simple (puede ser en papel o diagramador) mostrando cómo se conectan los componentes entre sí.**

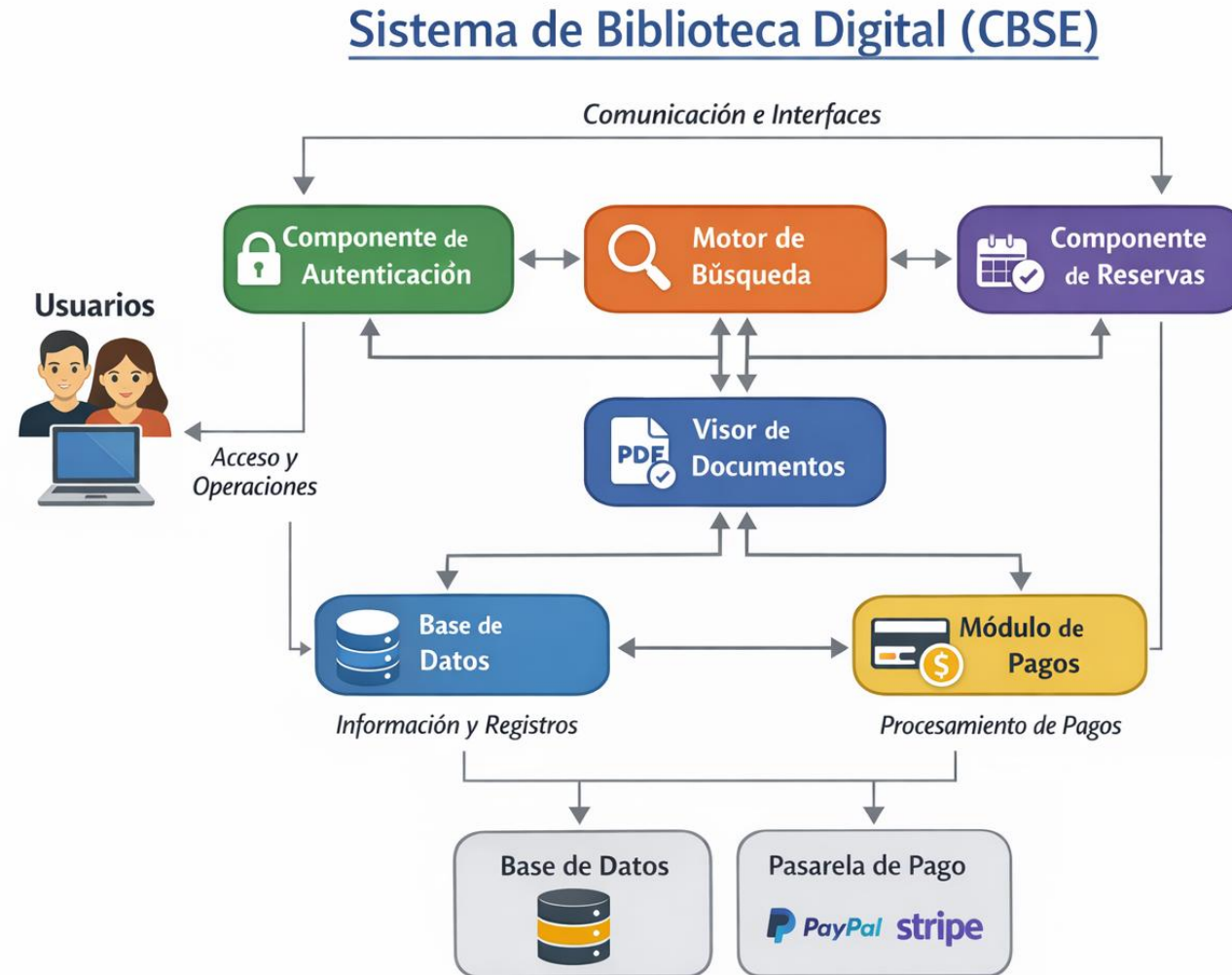
Indicar las interfaces principales (ejemplo: el componente de pagos se conecta con el de reservas).

#### Discusión y presentación (5 min)

Cada grupo explica brevemente:

- Qué componentes eligieron.
- Qué modelo de componentes usaron.
- Cómo se integran para formar el sistema.

# Solución Conceptual



# Antes de Concluir la Sesión

Comparte tus ejercicios de clase

¿Qué aprendimos hoy?

# Tarea para el Viernes 8 de mayo

Lecturas Desarrollo de Software Basado en Componentes

<https://tognu.home.blog/wp-content/uploads/2020/02/cap17.pdf>